

2024面试真题之框架篇(超全10万字)

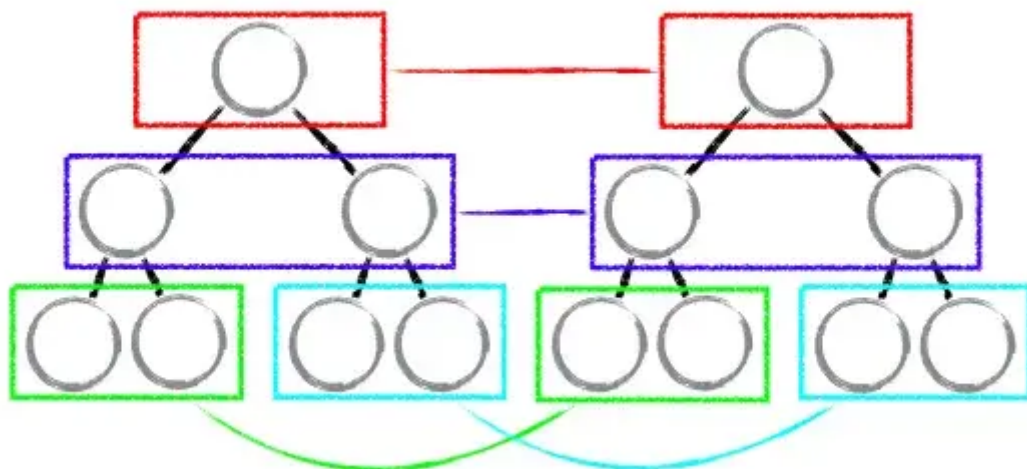
React Diff

在 React 中，diff算法 需要与 虚拟DOM 配合才能发挥出真正的威力。React 会使用 diff 算法计算出 虚拟DOM 中真正发生变化的部分，并且只会针对该部分进行 dom 操作，从而避免了对页面进行大面积的更新渲染，减小性能的开销。

React diff算法

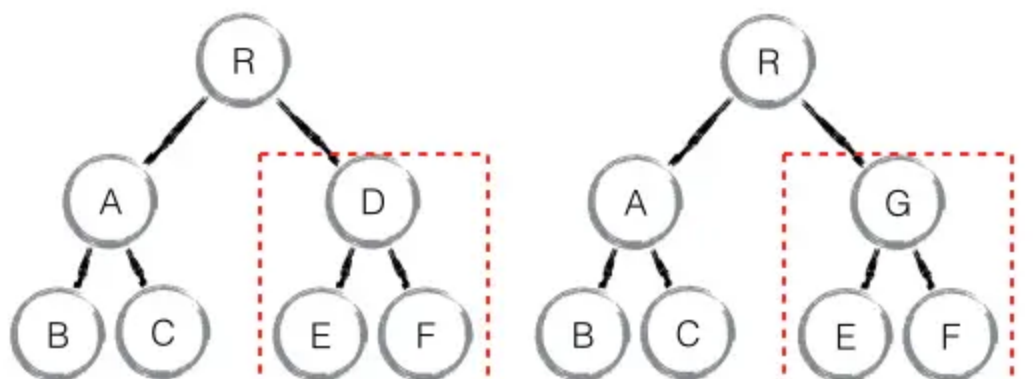
在 传统的diff算法 中复杂度会达到 $O(n^3)$ 。React 中定义了三种策略，在对比时，根据策略只需遍历一次树就可以完成对比，将复杂度降到了 $O(n)$ ：

1. **tree diff**: 在两个树对比时，只会比较同一层级的节点，会忽略掉跨层级的操作



2. **component diff**: 在对比两个组件时，首先会判断它们两个的类型是否相同

- 如果不是，则将该组件判断为 **dirty component**，从而替换整个组件下的所有子节点



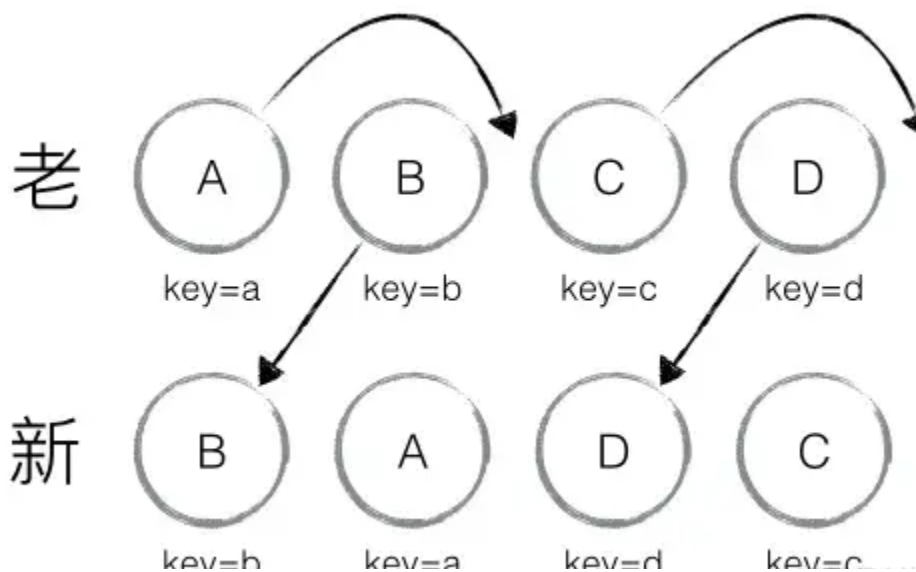
3. **element diff**: 对于同一层级的一组节点，会使用具有 唯一性的key 来区分是否需要创建，删除，或者是移动。

Element Diff

当节点处于同一层级时, `React diff` 提供了三种节点操作, 分别为:

1. `INSERT_MARKUP` (插入)
 - 新的 `component` 类型不在老集合里, 即是全新的节点, 需要对新节点执行**插入操作**
2. `MOVE_EXISTING` (移动)
 - 在老集合有新 `component` 类型, 且 `element` 是可更新的类型, 这种情况下 `prevChild = nextChild`, 就需要做移动操作, 可以复用以前的 DOM 节点。
3. `REMOVE_NODE` (删除)
 - 老 `component` 类型, 在新集合里也有, 但对应的 `element` 不同则不能直接复用和更新, 需要执行**删除操作**,
 - 或者老 `component` 不在新集合里的, 也需要执行**删除操作**

存在如下结构:



新老集合进行 `diff` 差异化对比, 通过 `key` 发现新老集合中的节点都是相同的节点, 因此无需进行节点 `删除和创建`, 只需要将老集合中节点的位置进行移动, 更新为新集合中节点的位置, 此时 `React` 给出的 `diff` 结果为: **B、D 不做任何操作, A、C 进行移动操作**

- 首先对新集合的节点进行循环遍历, `for (name in nextChildren)`,
- 通过**唯一 key** 可以判断新老集合中**是否存在相同的节点**, `if (prevChild === nextChild)`
- 如果存在**相同节点**, 则进行**移动操作**
- 但在移动前需要将当前节点在老集合中的位置与 `lastIndex` 进行比较,
 - `if (child._mountIndex < lastIndex)`, 则进行**节点移动操作**, 否则不执行该操
 - `lastIndex` 一直在更新, 表示访问过的节点在老集合中最右的位置 (即最大的位置),

- 如果新集合中当前访问的节点比 `lastIndex` 大，说明当前访问节点在老集合中就比较上一个节点位置靠后，则该节点不会影响其他节点的位置，因此不用添加到差异队列中，即不执行移动操作
- 只有当访问的节点比 `lastIndex` 小时，才需要进行移动操作。

当完成新集合中所有节点 `diff` 时，最后还需要对老集合进行循环遍历，判断是否存在新集合中没有但老集合中仍存在的节点，发现存在这样的节点 `x`，因此删除节点 `x`，到此 `diff` 全部完成。

setState同步异步问题

18.x之前版本

如果直接在 `setState` 后面获取 `state` 的值是获取不到的。

- 在 `React` 内部机制能检测到的地方，`setState` 就是异步的；
- 在 `React` 检测不到的地方，例如 原生事件 `addEventListener`，`setInterval`，`setTimeout`，`setState` 就是同步更新的

`setState` 并不是单纯的异步或同步，这其实与调用时的环境相关

- 在合成事件 和 生命周期钩子(除`componentDidUpdate`)中，`setState` 是"异步"的；
- 在 原生事件 和 `setTimeout` 中，`setState` 是同步的，可以马上获取更新后的值；

批量更新

多个顺序的 `setState` 不是同步地一个一个执行滴，会一个一个加入队列，然后最后一起执行。在 合成事件 和 生命周期钩子 中，`setState` 更新队列时，存储的是 合并状态 (`Object.assign`)。因此前面设置的 `key` 值会被后面所覆盖，最终只会执行一次更新。

异步现象原因

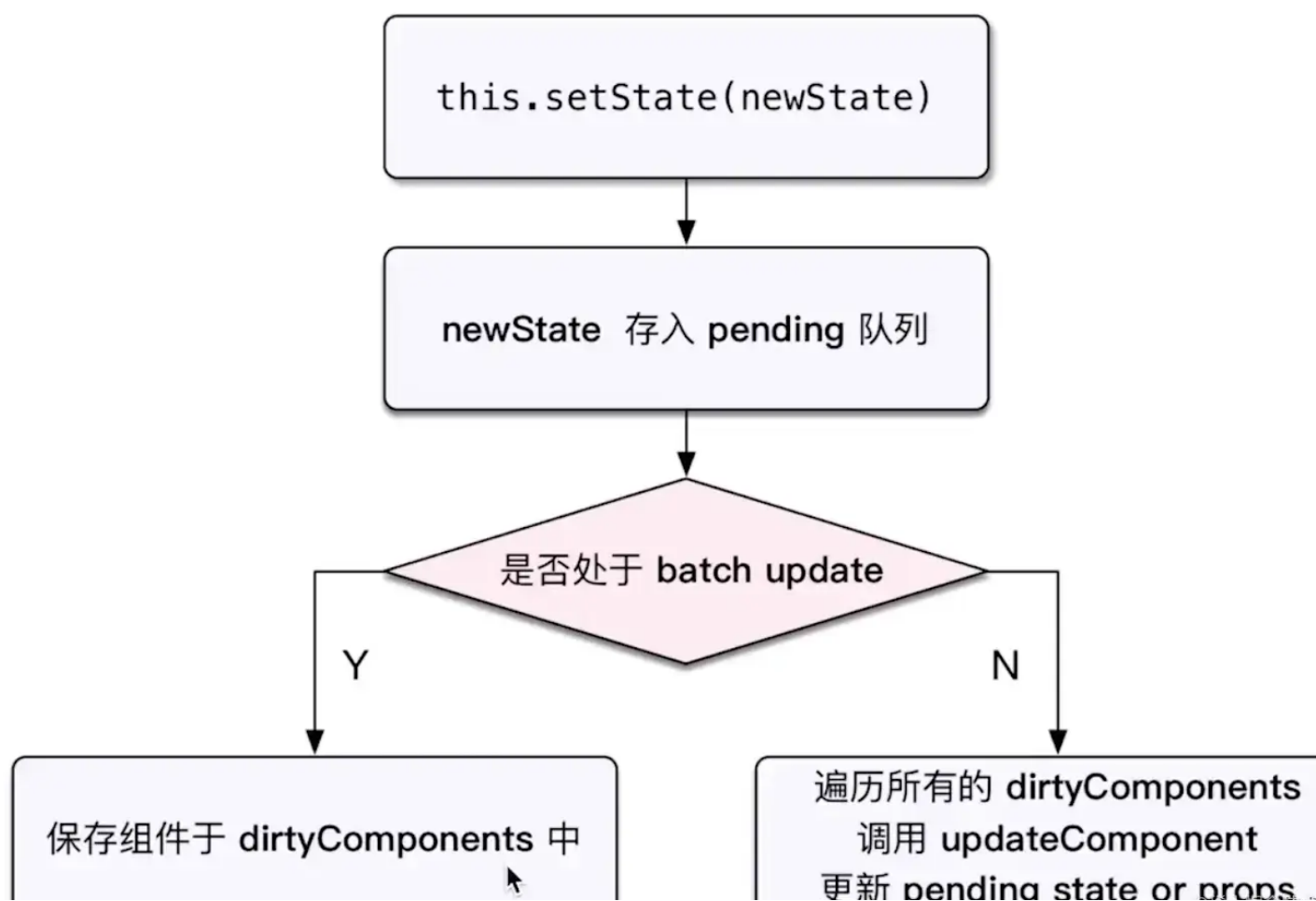
`setState` 的“异步”并不是说内部由异步代码实现，其实本身执行的过程和代码都是同步的，只是合成事件和生命钩子函数的调用顺序在更新之前，导致在 合成事件 和 钩子函数 中没法立马拿到更新后的值，形成了所谓的“异步”，当然可以通过第二个参数 `setState(partialState, callback)` 中的 `callback` 拿到更新后的结果。

`setState` 并非真异步，只是看上去像异步。在源码中，通过 `isBatchingUpdates` 来判断 `setState` 调用流程：

- 调用 `this.setState(newState)`
- 将新状态 `newState` 存入 `pending` 队列
- 判断是否处于 `batch Update` (`isBatchingUpdates`是否为true)

- `isBatchingUpdates=true`，保存组件于 `dirtyComponents` 中，走异步更新流程，合并操作，延迟更新；
- `isBatchingUpdates=false`，走**同步过程**。遍历所有的 `dirtyComponents`，调用 `updateComponent`，更新 `pending state or props`

setState 主流程



为什么直接修改this.state无效

`setState` 本质是通过一个队列机制实现 `state` 更新的。执行 `setState` 时，会将需要更新的 `state` 合并后放入**状态队列**，而不会立刻更新 `state`，队列机制可以批量更新 `state`。

如果不通过 `setState` 而直接修改 `this.state`，那么这个 `state` 不会放入状态队列中，下次调用 `setState` 时对状态队列进行合并时，会忽略之前直接被修改的 `state`，这样我们就无法合并了，而且实际也没有把你想要的 `state` 更新上去

React18

在 `v18` 之前只在事件处理函数中实现了批处理，在 `v18` 中所有更新都将自动批处理，包括 `promise` 链、`setTimeout` 等异步代码以及原生事件处理函数

React 18新特性

React 从 `v16` 到 `v18` 主打的特性包括三个变化：

- `v16`: `Async Mode` (异步模式)
- `v17`: `Concurrent Mode` (并发模式)
- `v18`: `Concurrent Render` (并发更新)

React 中 `Fiber` 树的更新流程分为两个阶段 `render` 阶段和 `commit` 阶段。

1. 组件的 `render` 函数执行时称为 `render` (本次更新需要做哪些变更)，纯 js 计算；
2. 而将 `render` 的结果渲染到页面的过程称为 `commit` (变更到真实的宿主环境中，在浏览器中就是操作 `DOM`)。

在 `Sync` 模式下，`render` 阶段是一次性执行完成；而在 `Concurrent` 模式下 `render` 阶段可以被拆解，每个时间片内执行一部分，直到执行完毕。由于 `commit` 阶段有 `DOM` 的更新，不可能让 `DOM` 更新到一半中断，必须一次性执行完毕。

React 并发新特性

并发渲染机制 `concurrent rendering` 的目的：根据用户的设备性能和网速对渲染过程进行适当的调整，保证 `React` 应用在长时间的渲染过程中依旧保持可交互性，避免页面出现卡顿或无响应的情况，从而提升用户体验。

1. 新 root API
 - 通过 `createRoot` Api 手动创建 `root` 节点。
2. 自动批处理优化 Automatic batching
 - `React` 将多个状态更新分组到一个重新渲染中以获得更好的性能。（将多次 `setstate` 事件合并）
 - 在 `v18` 之前只在事件处理函数中实现了批处理，在 `v18` 中所有更新都将自动批处理，包括 `promise` 链、`setTimeout` 等异步代码以及原生事件处理函数。
 - 想退出自动批处理立即更新的话，可以使用 `ReactDOM.flushSync()` 进行包裹
3. `startTransition`
 - 可以用来降低渲染优先级。分别用来包裹计算量大的 `function` 和 `value`，降低优先级，减少重复渲染次数。
 - `startTransition` 可以指定 UI 的渲染优先级，哪些需要实时更新，哪些需要延迟更新

- hook 版本的 `useTransition`，接受传入一个**毫秒的参数**用来修改最迟更新时间，返回一个过渡期的 `pending` 状态和 `startTransition` 函数。

4. `useDefferdValue`

- 通过 `useDefferdValue` 允许变量延时更新，同时接受一个可选的延迟更新的最大值。
`React` 将尝试尽快更新延迟值，如果在给定的 `timeoutMs` 期限内未能完成，它将强制更新。
- `const defferValue = useDeferredValue(value, { timeoutMs: 1000 })`
- `useDefferdValue` 能够很好的**展现并发渲染时优先级调整的特性**，可以用于**延迟计算逻辑比较复杂的状态**，让其他组件优先渲染，等待这个状态更新完毕之后再渲染。

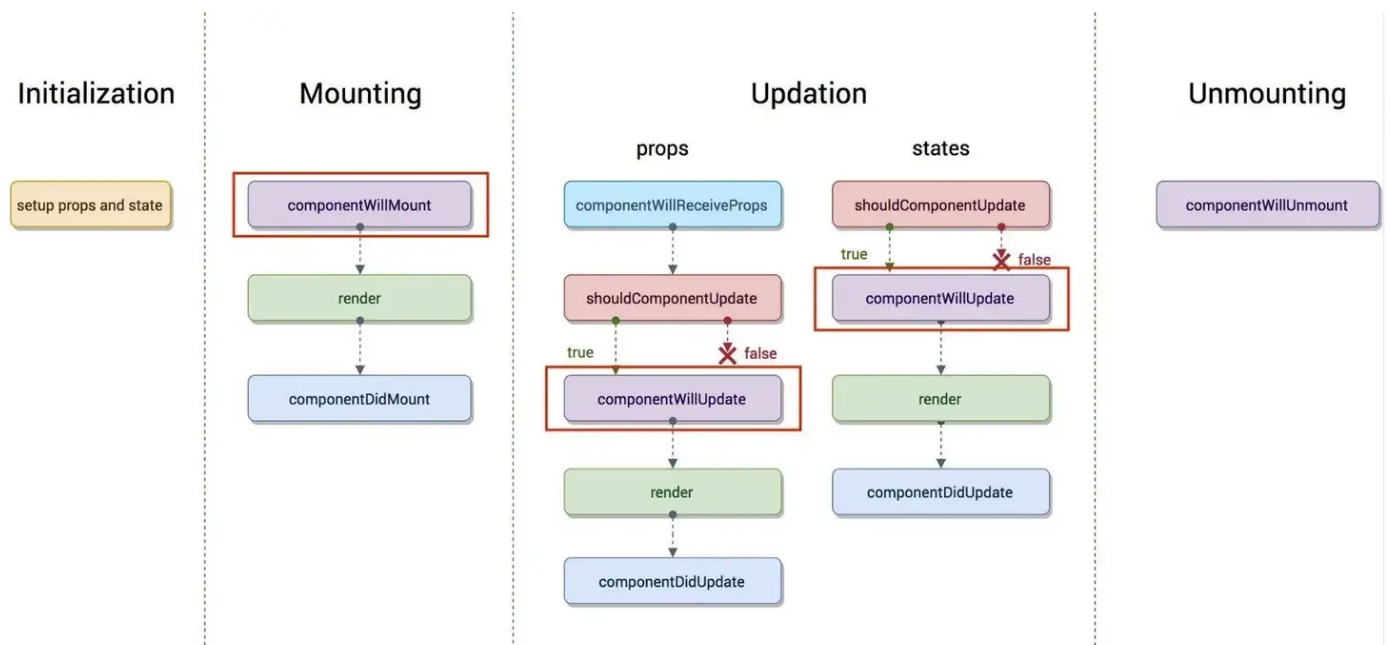
React 生命周期

生命周期 `React` 的生命周期主要有两个比较大的版本，分别是

- `v16.0` 前
- `v16.4` 两个版本

的生命周期。

v16.0前



总共分为**四大阶段**：

- {初始化| Intialization}
- {挂载| Mounting}
- {更新| Update}
- {卸载| Unmounting}

Intialization(初始化)

在初始化阶段,会用到 `constructor()` 这个构造函数,如:

```
1 constructor(props) {  
2   super(props);  
3 }
```

- `super` 的作用
 - 用来调用基类的构造方法(`constructor()`),
 - 也将父组件的 `props` 注入给子组件,供子组件读取
 - 初始化操作,定义 `this.state` 的初始内容
 - 只会执行一次
-

Mounting(挂载) (3个)

1. `componentWillMount` : 在组件挂载到 `DOM` 前调用
 - 这里面的调用的 `this.setState` 不会引起组件的重新渲染,也可以把写在这边的内容提到 `constructor()`,所以在项目中很少。
 - 只会调用一次
 2. `render` : 渲染
 - 只要 `props` 和 `state` 发生改变(无论值是否有变化,两者的重传递和重赋值,都可以引起组件重新 `render`),都会重新渲染 `render`。
 - `return` : 是必须的,是一个 `React` 元素,不负责组件实际渲染工作,由 `React` 自身根据此元素去渲染出 `DOM`。
 - `render` 是纯函数,不能执行 `this.setState`。
 3. `componentDidMount` : 组件挂载到 `DOM` 后调用
 - 调用一次
-

Update(更新) (5个)

1. `componentWillReceiveProps(nextProps)` : 调用于 `props` 引起的组件更新过程中
 - `nextProps` : 父组件传给当前组件新的 `props`

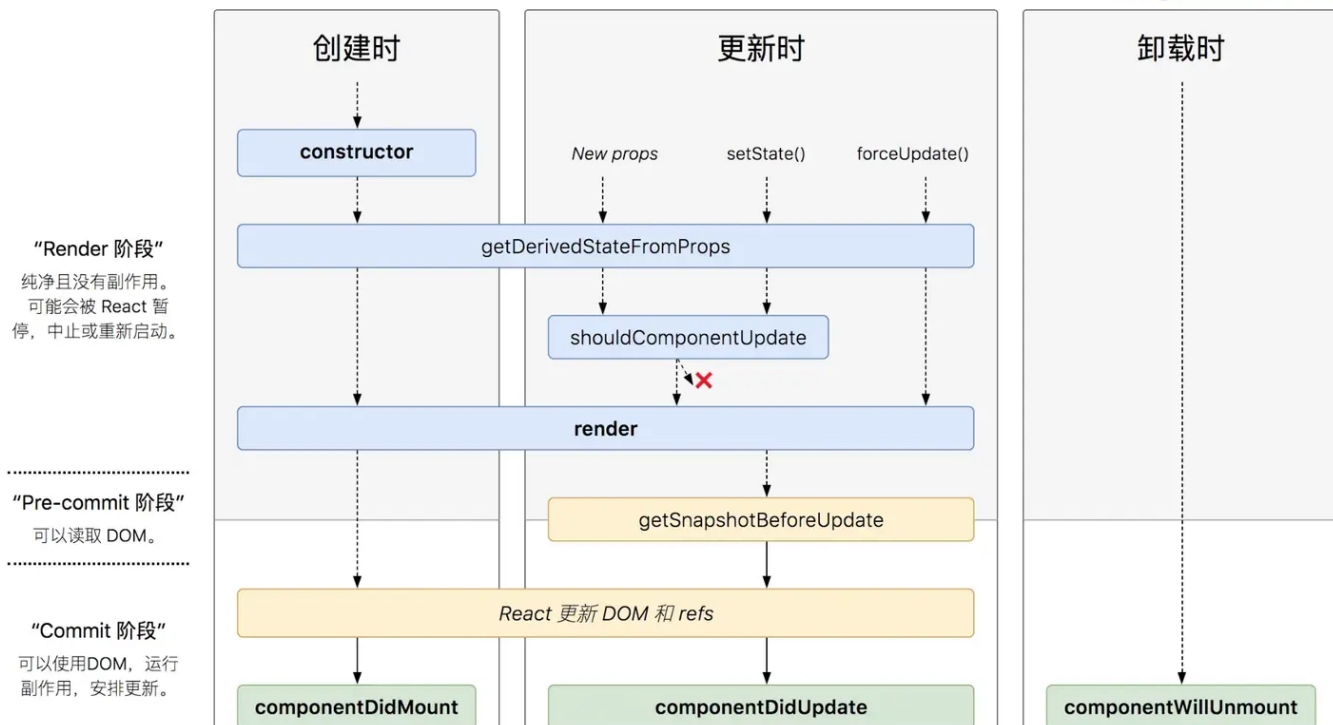
- 可以用 `nextProps` 和 `this.props` 来查明重传 `props` 是否发生改变（原因：不能保证父组件重传的 `props` 有变化）
 - 只要 `props` 发生变化就会，引起调用
2. `shouldComponentUpdate(nextProps, nextState)`：用于性能优化
- `nextProps`：当前组件的 `this.props`
 - `nextState`：当前组件的 `this.state`
 - 通过比较 `nextProps` 和 `nextState` ,来判断当前组件是否有必要继续执行更新过程。
 - 返回 `false`：表示停止更新，用于减少组件的不必要渲染，优化性能
 - 返回 `true`：继续执行更新
 - 像 `componentWillReceiveProps()` 中执行了 `this.setState`，更新了 `state`，但在 `render` 前(如 `shouldComponentUpdate`，`componentWillUpdate`)，`this.state` 依然指向更新前的 `state`，不然 `nextState` 及当前组件的 `this.state` 的对比就一直是 `true` 了
3. `componentWillUpdate(nextProps, nextState)`：组件更新前调用
- 在 `render` 方法前执行
 - 由于组件更新就会调用，所以一般很少使用
4. `render`：重新渲染
5. `componentDidUpdate(prevProps, prevState)`：组件更新后被调用
- `prevProps`：组件更新前的 `props`
 - `prevState`：组件更新前的 `state`
 - 可以操作组件更新的DOM
-

Unmounting(卸载) (1个)

`componentWillUnmount`：组件被卸载前调用

可以在这里执行一些**清理工作**，比如清除组件中使用的**定时器**，清除 `componentDidMount` 中**手动创建的DOM元素**等，以避免引起内存泄漏

React v16.4



与 v16.0 的生命周期相比

- 新增了 -- (两个 `getXX`)
 - `getDerivedStateFromProps`
 - `getSnapshotBeforeUpdate`
- 取消了 -- (三个 `componentWillXX`)
 - `componentWillMount`、
 - `componentWillReceiveProps`、
 - `componentWillUpdate`

getDerivedStateFromProps

`getDerivedStateFromProps(prevProps, prevState)`：组件创建和更新时调用的方法

- `prevProps`：组件更新前的 `props`
- `prevState`：组件更新前的 `state`

在 React v16.3 中，在创建和更新时，只能是由父组件引发才会调用这个函数，在 React v16.4 改为无论是 `Mounting` 还是 `Updating`，全部都会调用。

是一个静态函数，也就是这个函数不能通过 `this` 访问到 `class` 的属性。

如果 `props` 传入的内容不需要影响到你的 `state`，那么就需要返回一个 `null`，这个返回值是必须的，所以尽量将其写到函数的末尾。

在组件创建时和更新时的 `render` 方法之前调用，它应该

- 返回一个对象来更新状态
- 或者返回 `null` 来不更新任何内容

getSnapshotBeforeUpdate

`getSnapshotBeforeUpdate(prevProps, prevState)` : `Updating` 时的函数，在render之后调用

- `prevProps` : 组件更新前的 `props`
- `prevState` : 组件更新前的 `state`

可以读取，但无法使用DOM的时候，在组件可以在可能更改之前从 `DOM` 捕获一些信息（例如滚动位置）

返回的任何值都将作为参数传递给 `componentDidUpdate()`

Note

在 `17.0` 的版本，官方彻底废除

- `componentWillMount` 、
- `componentWillReceiveProps` 、
- `componentWillUpdate`

Hook的相关知识点

`react-hooks` 是 `React 16.8` 的产物，给函数式组件赋上了生命周期。

React v16.8中的hooks

useState

`useState` : 定义变量，可以理解为他是类组件中的 `this.state` 使用：

```
1 const [state, setState] = useState(initialState);
```

- `state` : 目的是提供给 UI，作为渲染视图的数据源
- `setState` : 改变 `state` 的函数，可以理解为 `this.setState`
- `initialState` : 初始默认值

`useState` 有点类似于 `PureComponent` ,会进行一个比较浅的比较,如果是对象的时候直接传入并不会更新。

解决传入对象的问题

使用 `useImmer` 替代 `useState`。

`immer.js` 这个库,是基于 `proxy` 拦截 `getter` 和 `setter` 的能力,让我们可以很方便的通过修改对象本身,创建新的对象。

`React` 通过 `Object.is` 函数比较 `props` ,也就是说对于引用一致的对象,react是不会刷新视图的,这也是为什么我们不能直接修改调用 `useState` 得到的 state 来更新视图,而是要通过 `setState` 刷新视图,通常,为了方便,我们会使用 `es6` 的 `spread` 运算符构造新的对象(浅拷贝)。

对于嵌套层级多的对象,使用 `spread` 构造新的对象写起来心智负担很大,也不易于维护常规的处理方式是对数据进行 `deepClone` ,但是这种处理方式针对结构简单的数据来讲还算OK,但是遇到大数据的话,就不够优雅了。

所以,我们可以直接使用 `useImmer` 这个语法糖来进一步简化调用方式

```
1  const [state,setState] = useImmer({
2    a: 1,
3    b: {
4      c: [1,2]
5      d: 2
6    },
7  });
8
9  setState(prev => {
10    prev.b.c.push(3);
11  })
```

useEffect

`useEffect` : 副作用,你可以理解为是类组件的生命周期,也是我们最常用的钩子

副作用 (Side Effect): 是指 `function` 做了和本身运算返回值无关的事,如请求数据、修改全局变量,打印、数据获取、设置订阅以及手动更改 `React` 组件中的 `DOM` 都属于副作用操作

1. 不断执行

- 当 `useEffect` 不设立第二个参数时,无论什么情况,都会执行

2. 根据依赖值改变

- 设置 `useEffect` 的第二个值

useContext

`useContext`：上下文，类似于 `Context`：其本意就是设置全局共享数据，使所有组件可跨层级实现数据共享

`useContext` 的参数一般是由 `createContext` 的创建，通过 `xxContext.Provider` 包裹的组件，才能通过 `useContext` 获取对应的值

存在的问题及解决方案

`useContext` 是 `React` 官方推荐的共享状态的方式，然而在需要共享状态的组件非常多的情况下，这有着严重的性能问题，例如有A/B组件，A组件只更新 `state.a`，并没有用到 `state.b`，B组件更新 `state.b` 的时候A组件也会刷新，在组件非常多的情况下，就卡死了，用户体验非常不好。

解决上述问题，可以使用 `react-tracked` 这个库，它拥有和 `useContext` 差不多的 api，但基于 `proxy` 和组件内部的 `useForceUpdate` 做到了自动化的追踪，可以精准更新每个组件，不会出现修改大的 `state`，所有组件都刷新的情况。

useReducer

`useReducer`：它类似于 `redux` 功能的api

```
1 const [state, dispatch] = useReducer(reducer, initialArg, init);
```

- `state`：更新后的 `state` 值
- `dispatch`：可以理解为和 `useState` 的 `setState` 一样的效果
- `reducer`：可以理解为 `redux` 的 `reducer`
- `initialArg`：初始值
- `init`：惰性初始化

useMemo

`useMemo`：与 `memo` 的理念上差不多，都是判断是否满足当前的限定条件来决定是否执行 `callback` 函数，而 `useMemo` 的第二个参数是一个数组，通过这个数组来判定是否执行回调函数

当一个父组件中调用了一个子组件的时候，父组件的 `state` 发生变化，会导致父组件更新，而子组件虽然没有发生改变，但也会进行更新。

只要父组件的状态更新，**无论有没有对子组件进行操作，子组件都会进行更新**，`useMemo` 就是为了防止这点而出现的。

useCallback

`useCallback` 与 `useMemo` 极其类似,唯一不同的是

- `useMemo` 返回的是**函数运行的结果**
- 而 `useCallback` 返回的是**函数**
 - 这个函数是父组件传递子组件的一个函数，**防止做无关的刷新**，
 - 其次，这个子组件必须配合 `React.memo` ,否则不但不会提升性能，还有可能降低性能

存在的问题及解决方案

一个很常见的误区是为了心理上的性能提升把函数通通使用 `useCallback` 包裹，在大多数情况下，`javascript` **创建一个函数的开销是很小的**，哪怕每次渲染都重新创建，也不会有太大的性能损耗，真正的性能损耗在于，很多时候 `callback` 函数是组件 `props` 的一部分，因为每次渲染的时候都会重新创建 `callback` 导致函数引用不同，所以触发了组件的重渲染。然而一旦函数使用 `useCallback` 包裹，则要面对声明依赖项的问题，对于一个内部捕获了很多 `state` 的函数，写依赖项非常容易写错，因此引发 `bug`。

所以，在大多数场景下，我们应该只在需要维持函数引用的情况下使用 `useCallback`。

```
1  const [userText, setUserText] = useState("");
2  const handleUserKeyPress = useCallback(event => {
3      // do something here
4  }, []);
5
6  useEffect(() => {
7      window.addEventListener("keydown", handleUserKeyPress);
8      return () => {
9          window.removeEventListener("keydown", handleUserKeyPress);
10     };
11 }, [handleUserKeyPress]);
12
13 return (
14     <div>
15         {userText}
16     </div>
17 );
```

在组件卸载的时候移除 `event listener callback`，因此需要保持 `event handler` 的引用，所以这里需要使用 `useCallback` 来保持引用不变。

使用 `useCallback`，我们又会**面临声明依赖项的问题**，这里我们可以使用 `ahook` 中的 `useMemoizedFn` 的方式，既能保持引用，又不用声明依赖项。

```
1 const [state, setState] = useState('');
2 // func 地址永远不会变化
3 const func = useMemoizedFn(() => {
4   console.log(state);
5 });
```

useRef

`useRef`：可以获取当前元素的**所有属性**，并且返回一个可变的 `ref对象`，并且这个对象只有 `current属性`，可设置 `initialValue`

1. 通过 `useRef` 获取对应的 `React元素` 的属性值
2. 缓存数据

useImperativeHandle

`useImperativeHandle`：可以让你在使用 `ref` 时**自定义暴露给父组件的实例值**

```
1 useImperativeHandle(ref, createHandle, [deps])
```

- `ref`： `useRef` 所创建的 `ref`
- `createHandle`：**处理的函数**，返回值作为暴露给父组件的 `ref` 对象。
- `deps`：**依赖项**，依赖项更改形成新的 `ref` 对象。

`useImperativeHandle` 和 `forwardRef` 配合使用

```
1 function FancyInput(props, ref) {
2   const inputRef = useRef();
3   useImperativeHandle(ref, () => ({
4     focus: () => {
5       inputRef.current.focus();
6     }
9   }

```

```
7   }));  
8   return <input ref={inputRef} ... />;  
9 }  
10 FancyInput = forwardRef(FancyInput);
```

在父组件中，可以渲染 `<FancyInput ref={inputRef} />` 并可以通过父组件的 `inputRef` 对子组件中的 `input` 进行处理。

- `inputRef.current.focus()`

useLayoutEffect

`useLayoutEffect`：与 `useEffect` 基本一致，不同的地方时，`useLayoutEffect` 是同步要注意的是 `useLayoutEffect` 在 DOM 更新之后，浏览器绘制之前，这样做的好处是可以更加方便的**修改 DOM，获取 DOM 信息**，这样浏览器只会绘制一次，所以 `useLayoutEffect` 在 `useEffect` 之前执行如果是 `useEffect` 的话，`useEffect` 执行在浏览器绘制视图之后，如果在此时改变 DOM，有可能导致浏览器再次回流和重绘。

除此之外 `useLayoutEffect` 的 `callback` 中代码执行会阻塞浏览器绘制

useDebugValue

`useDebugValue`：可用于在 `React` 开发者工具中显示自定义 `hook` 的标签

React v18中的hooks

useSyncExternalStore

`useSyncExternalStore`：是一个推荐用于**读取和订阅外部数据源**的 `hook`，其方式与选择性的 `hydration` 和时间切片等并发渲染功能兼容

```
1 const state = useSyncExternalStore(  
2   subscribe,  
3   getSnapshot[, getServerSnapshot]  
4 )
```

- `subscribe`：订阅函数，用于注册一个回调函数，**当存储值发生更改时被调用**。此外，`useSyncExternalStore` 会通过带有记忆性的 `getSnapshot` 来判别数据是否发生变化，如果发生变化，那么会**强制更新数据**。

- `getSnapshot` : 返回当前存储值的函数。必须返回缓存的值。如果 `getSnapshot` 连续多次调用, 则必须返回相同的确切值, 除非中间有存储值更新。
- `getServerSnapshot` : 返回服务端(hydration模式下)渲染期间使用的存储值的函数

useTransition

`useTransition` :

- 返回一个**状态值**表示过渡任务的等待状态,
- 以及一个启动该过渡任务的函数。

过渡任务 在一些场景中, 如: `输入框`、`tab切换`、`按钮` 等, 这些任务需要视图上立刻做出响应, 这些任务可以称之为**立即更新的任务**

但有的时候, 更新任务并不是那么紧急, 或者说要去请求数据等, 导致新的状态不能立马更新, 需要用 `loading...` 的等待状态, 这类任务就是过渡任务

```
1 const [isPending, startTransition] = useTransition();
```

- `isPending` : **过渡状态的标志**, 为 `true` 时是等待状态
- `startTransition` : 可以将里面的任务变成过渡任务

useDeferredValue

`useDeferredValue` : 接受一个值, 并返回该值的新副本, 该副本将**推迟**到更紧急地更新之后。

如果当前渲染是一个紧急更新的结果, 比如用户输入, `React` 将**返回之前的值**, 然后在**紧急渲染完成后渲染新的值**。

也就是说 `useDeferredValue` 可以让状态滞后派生。

```
1 const deferredValue = useDeferredValue(value);
```

- `value` : 可变的值, 如 `useState` 创建的值
- `deferredValue` : 延时状态

useTransition和useDeferredValue做个对比

- 相同点: `useDeferredValue` 和 `useTransition` 一样, 都是**过渡更新任务**

- 不同点： `useTransition` 给的是一个状态，而 `useDeferredValue` 给的是一个值

useInsertionEffect

`useInsertionEffect`：与 `useLayoutEffect` 一样，但它在所有 DOM 突变之前同步触发
在执行顺序上 `useInsertionEffect``useLayoutEffect``useEffect`

`useInsertionEffect` 应仅限于 `css-in-js` 库作者使用。

优先考虑使用 `useEffect` 或 `useLayoutEffect` 来替代。

useId

`useId`：是一个用于生成横跨服务端和客户端的稳定的唯一 ID 的同时避免 hydration 不匹配的 hook。

ref能否拿到函数组件的实例

使用 forwardRef

将 `input` 单独封装成一个组件 `TextInput`。

```
1 const TextInput = React.forwardRef((props, ref) => {  
2   return <input ref={ref}></input>  
3 })
```

用 `TextInputWithFocusButton` 调用它

```
1 function TextInputWithFocusButton() {  
2   // 关键代码  
3   const inputEl = useRef(null);  
4   const onClick = () => {  
5     // 关键代码，`current` 指向已挂载到 DOM 上的文本输入元素  
6     inputEl.current.focus();  
7   };  
8   return (  
9     <>  
10    // 关键代码  
11    <TextInput ref={inputEl}></TextInput>  
12    <button onClick={onClick}>Focus the input</button>
```

```
13     </>
14   );
15 }
```

useImperativeHandle

有时候，我们可能**不想将整个子组件暴露给父组件**，而只是暴露出父组件需要的值或者方法，这样可以**让代码更加明确**。而 `useImperativeHandle` Api就是帮助我们做这件事的。

```
1  const TextInput = forwardRef((props, ref) => {
2    const inputRef = useRef();
3    // 关键代码
4    useImperativeHandle(ref, () => ({
5      focus: () => {
6        inputRef.current.focus();
7      }
8    }));
9    return <input ref={inputRef} />
10  })
11
12
13  function TextInputWithFocusButton() {
14    // 关键代码
15    const inputEl = useRef(null);
16    const onClick = () => {
17      // 关键代码，`current` 指向已挂载到 DOM 上的文本输入元素
18      inputEl.current.focus();
19    };
20    return (
21      <>
22        // 关键代码
23        <TextInput ref={inputEl}></TextInput>
24        <button onClick={onClick}>
25          Focus the input
26        </button>
27      </>
28    );
29  }
30
```

也可以使用 `current.focus()` 来做 `input` 聚焦。

这里要注意的是，子组件 `TextInput` 中的 `useRef` 对象，只是用来获取 `input` 元素的，大家不要和父组件的 `useRef` 混淆了。

useCallback vs useMemo的区别

useMemo

```
1 const memoizedValue = useMemo(  
2   () => computeExpensiveValue(a, b),  
3   [a, b]  
4 );
```

`useMemo` 与 `memo` 的理念上差不多，都是判断是否满足**当前的限定条件**来决定是否执行 `callback` 函数，而 `useMemo` 的第二个参数是一个**数组**，通过这个数组来判定是否执行回调函数。当一个父组件中调用了一个子组件的时候，父组件的 `state` 发生变化，会导致**父组件更新**，而子组件虽然没有发生改变，但也会进行更新。

只要父组件的状态更新，无论有没有对子组件进行操作，子组件都会进行更新，`useMemo` 就是为了防止这点而出现的。

useCallback

`useCallback` 可以理解为 `useMemo` 的语法糖

```
1 const memoizedCallback = useCallback(  
2   + () => {  
3     doSomething(a, b);  
4   + },  
5   [a, b],  
6 );
```

`useCallback` 与 `useMemo` 极其类似,唯一不同的是

- `useMemo` 返回的是**函数运行的结果**
- 而 `useCallback` 返回的是**函数**
 - 这个函数是父组件传递子组件的一个函数，防止做无关的刷新，
 - 其次，这个子组件必须配合 `React.memo` ,否则不但不会提升性能，还有可能降低性能

React.memo

`memo`：结合了 `pureComponent` 纯组件和 `componentShouldUpdate()` 功能，会对传入的 `props` 进行一次对比，然后根据**第二个函数返回值**来进一步判断哪些 `props` 需要更新

要注意 `memo` 是一个高阶组件，函数式组件和类组件都可以使用。

`memo` 接收两个参数：

```
1 function MyComponent(props) {  
2  
3 }  
4 function areEqual(prevProps, nextProps) {  
5  
6 }  
7 export default React.memo(MyComponent, areEqual);
```

1. 第一个参数：**组件本身**，也就是要优化的组件
2. 第二个参数：`(pre, next) => boolean`，
 - `pre`：之前的数据
 - `next`：现在的数据
 - 返回一个布尔值
 - 若为 `true` 则不更新
 - 为 `false` 更新

memo的注意事项

`React.memo` 与 `PureComponent` 的区别：

- 服务对象不同：
 - `PureComponent` 服务于**类组件**，
 - `React.memo` 既可以服务于类组件，也可以服务与函数式组件，
 - `useMemo` 服务于函数式组件
- 针对的对象不同：
 - `PureComponent` 针对的是 `props` 和 `state`
 - `React.memo` 只能针对 `props` 来决定是否渲染

`React.memo` 的第二个参数的返回值与 `shouldComponentUpdate` 的返回值是**相反的**

- `React.memo` :返回 `true` 组件不渲染，返回 `false` 组件重新渲染。
- `shouldComponentUpdate` :返回 `true` 组件渲染，返回 `false` 组件不渲染

类组件和函数组件的区别

相同点

组件是 `React` 可复用的最小代码片段，它们会返回要在页面中渲染 `React` 元素，也正是基于这一点，所以在 `React` 中无论是函数组件，还是类组件，其实它们最终呈现的效果都是一致的。

不同点

设计思想

1. 类组件的根基是 `OOP` (面向对象编程)，所以它会有继承，有内部状态管理等
2. 函数组件的根基是 `FP` (函数式编程)

未来的发展趋势

`React` 团队从 `Facebook` 的实际业务场景触发，通过探索时间切片和并发模式，以及考虑性能的进一步优化和组件间更合理的代码拆分后，认为类组件的模式并不能很好地适应未来的趋势，它们给出了以下3个原因：

1. `this` 的模糊性
2. 业务逻辑耦合在生命周期中
3. `React` 的组件代码缺乏标准的拆分方式

componentWillUnmount在浏览器刷新后，会执行吗

不会。

如果实现，在刷新页面时进行数据处理。使用 `beforeunload` 事件。

还有一个 `navigator.sendBeacon()`

React 组件优化

1. 父组件刷新，而不波及子组件
2. 组件自己控制自己是否刷新
3. 减少波及范围，无关刷新数据不存入 `state` 中
4. 合并 `state` ,减少重复 `setState` 的操作

父组件刷新，而不波及子组件

1. 子组件自己判断是否需要更新,典型的的就是

- `PureComponent` ,
- `shouldComponentUpdate` ,
- `React.memo`

2. 父组件对子组件做个缓冲判断

使用PureComponent注意点

1. 父组件是函数组件，子组件用 `PureComponent` 时，匿名函数，箭头函数和普通函数都会重新声明

- 可以使用 `useMemo` 或者 `useCallback` , 利用他们缓冲一份函数，保证不会出现重复声明就可以了。

2. 类组件中不使用箭头函数，匿名函数

- `class` 组件中每一次刷新都会重复调用 `render` 函数，那么 `render` 函数中使用的匿名函数，箭头函数就会造成重复刷新的问题
- 处理方式- 换成普通函数

3. 在 `class` 组件的 `render` 函数中调用 `bind` 函数

- 把 `bind` 操作放在 `constructor` 中

shouldComponentUpdate

`class` 组件中使用 `shouldComponentUpdate` 是主要的优化方式，它不仅仅可以判断来自父组件的 `nextprops` , 还可以根据 `nextState` 和最新的 `nextContext` 来决定是否更新。

React.memo

`React.memo` 的规则是如果想要复用最后一次渲染结果，就返回 `true` , 不想复用就返回 `false` 。所以它和 `shouldComponentUpdate` 的正好相反，`false` 才会更新，`true` 就返回缓冲。

```
1 const Children = React.memo(function ({count}){
2   return (
3     <div>
4       只有父组件传入的值是偶数的时候才会更新
5       {count}
6     </div>
7   )
8 },(prevProps, nextProps)=>{
```

```
9     if(nextProps.count % 2 === 0){
10         return false;
11     }else{
12         return true;
13     }
14 })
```

使用 React.useMemo来实现对子组件的缓冲

子组件只关心 `count` 数据，当我们刷新 `name` 数据的时候，并不会触发刷新 `Children`子组件，实现了我们对组件的缓冲控制。

```
1 export default function Father () {
2     let [count, setCount] = React.useState(0);
3     let [name, setName] = React.useState(0);
4     const render = React.useMemo(
5         () =>
6             <Children count = {count}/>
7             , [count]
8     )
9     return (
10         <div>
11             <button onClick={() => setCount(++count)}>
12                 点击刷新count
13             </button>
14             <br/>
15             <button onClick={() => setName(++name)}>
16                 点击刷新name
17             </button>
18             <br/>
19             {"count"+count}
20             <br/>
21             {"name"+name}
22             <br/>
23             {render}
24         </div>
25     )
26 }
```

减少波及范围，无关刷新数据不存入state中

1. 无意义重复调用 `setState`，合并相关的 `state`

2. 和页面刷新无关的数据，不存入 `state` 中
 3. 通过存入 `useRef` 的数据中，避免父子组件的重复刷新
 4. 合并 `state` ,减少重复 `setState` 的操作
 - `ReactDOM.unstable_batchedUpdates` ;
 - 多个 `setState` 会合并执行一次。
-

React-Router实现原理

react-router-dom和react-router和history库三者什么关系

1. `history` 可以理解为 `react-router` 的核心，也是整个路由原理的核心，里面集成了 `popState` , `history.pushState` 等底层路由实现的原理方法
2. `react-router` 可以理解为是 `react-router-dom` 的核心，里面封装了 `Router` , `Route` , `Switch` 等核心组件,实现了从路由的改变到组件的更新的核心功能
3. `react-router-dom` ,在 `react-router` 的核心基础上，添加了用于跳转的 `Link` 组件，和 `history` 模式下的 `BrowserRouter` 和 `hash` 模式下的 `HashRouter` 组件等。
 - 所谓 `BrowserRouter` 和 `HashRouter` ，也只不过用了 `history` 库中 `createBrowserHistory` 和 `createHashHistory` 方法

单页面实现核心原理

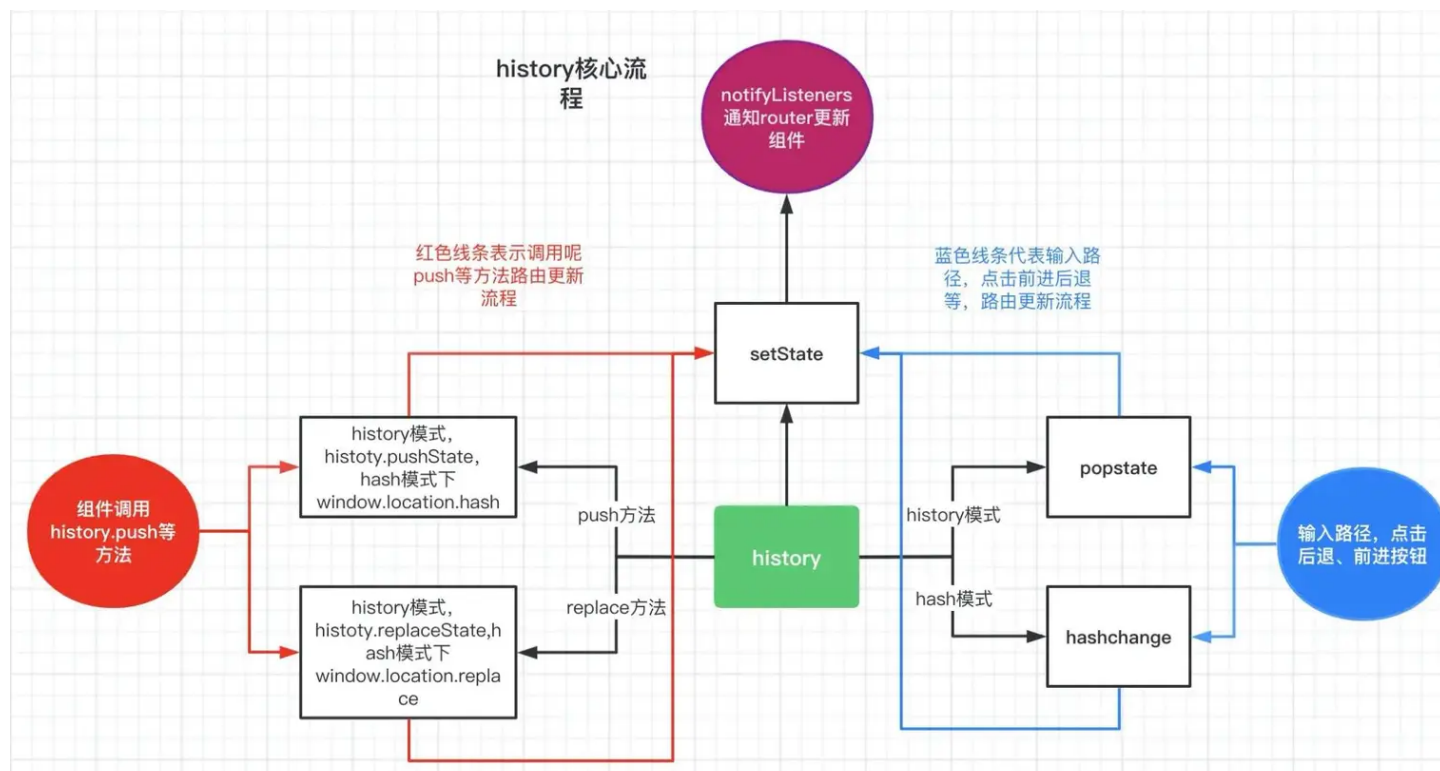
单页面应用路由实现原理是，切换 `url` ，监听 `url` 变化，从而渲染不同的页面组件。
主要的方式有 `history` 模式和 `hash` 模式。

history模式原理

1. 改变路由
 - `history.pushState(state,title,path)`
2. 监听路由
 - `window.addEventListener('popstate',function(e){ /* 监听改变 */})`

hash模式原理

1. 改变路由
 - 通过 `window.location.hash` 属性获取和设置 `hash` 值
2. 监听路由
 - `window.addEventListener('hashchange',function(e){ /* 监听改变 */})`



XXR

根据不同的构建、渲染过程有不同的优劣势和适用情况。

- 现代 UI 库加持下常用的 `CSR`、
- 具有更好 `SEO` 效果的 `SSR` (`SPR`)、
- 转换思路主打**构建时生成**的 `SSG`、
- 大架构视野之上的 `ISR`、`DPR`，
- 还有更少听到的 `NSR`、`ESR`。

CSR(Client Side Rendering)

页面托管服务器只需要对页面的访问请求响应一个如下的空页面

```

1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta charset="utf-8" />
5     <!-- metas -->
6     <title></title>
7     <link rel="shortcut icon" href="xxx.png" />
8     <link rel="stylesheet" href="xxx.css" />
9   </head>
10  <body>

```

```
11 <div id="root"><!-- page content --></div>
12 <script src="xxx/filterXss.min.js"></script>
13 <script src="xxx/x.chunk.js"></script>
14 <script src="xxx/main.chunk.js"></script>
15 </body>
16 </html>
```

页面中留出一个用于填充渲染内容的视图节点 (`div#root`), 并插入指向项目编译压缩后的

- `JS Bundle` 文件的 `script` 节点
- 指向 `CSS` 文件的 `link.stylesheet` 节点等。

浏览器接收到这样的文档响应之后, 会根据文档内的链接加载脚本与样式资源, 并完成以下几方面主要工作:

1. 执行脚本
2. 进行网络访问以获取在线数据
3. 使用 DOM API 更新页面结构
4. 绑定交互事件
5. 注入样式

以此完成整个渲染过程。

CSR 模式有以下几方面优点:

- UI 库支持
- 前后端分离
- 服务器负担轻

SSR (Server Side Rendering)

SSR 的概念, 即与 `CSR` 相对地, 在服务端完成大部分渲染工作, --- 服务器在响应站点访问请求的时候, 就已经渲染好可供呈现的页面。

像 `React`、`Vue` 这样的 UI 生态巨头, 其实都有一个关键的 `Virtual DOM` (or `VDOM`) 概念, 先自己建模处理视图表现与更新、再批量调 `DOM API` 完成视图渲染更新。这就带来了一种 `SSR` 方案:

`VDOM` 是自建模型, 是一种抽象的嵌套数据结构, 也就可以在 `Node` 环境 (或者说一切服务端环境) 下跑起来, 把原来的视图代码拿来在服务端跑, 通过 `VDOM` 维护, 再在最后拼接好字符串作为页面响应, 生成文档作为响应页面, 此时的页面内容已经基本生成完毕, 把逻辑代码、样式代码附上, 则可以实现完整的、可呈现页面的响应。

SSR优点

- 呈现速度和用户体验佳
- `SEO` 友好

SSR缺点

1. 引入成本高

- 将视图渲染的工作交给了服务器做，引入了新的概念和技术栈（如 Node）

2. 响应时间长

- SSR 在完成访问响应的时候需要做更多的计算和生成工作
- 关键指标 `TTFB` (`Time To First Byte`) 将变得更大

3. 首屏交互不佳

- 虽然 SSR 可以让页面请求响应后更快在浏览器上渲染出来
- 但在首帧出现，需要客户端加载激活的逻辑代码（如事件绑定）还没有初始化完毕的时候，其实是不可交互的状态

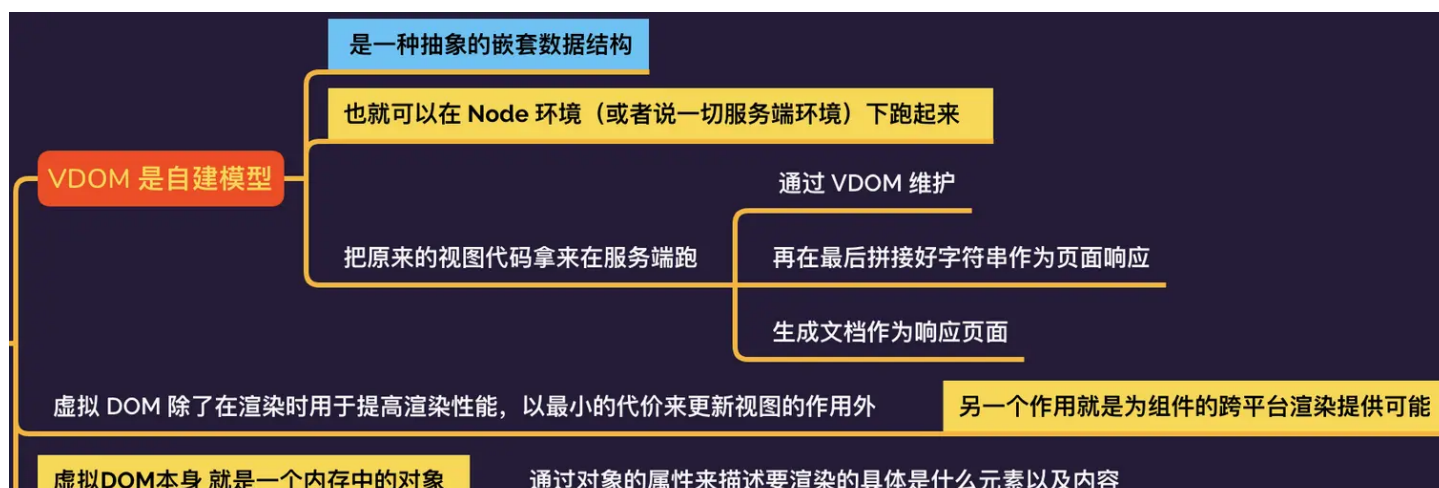
SSR-React 原理

1. VDOM

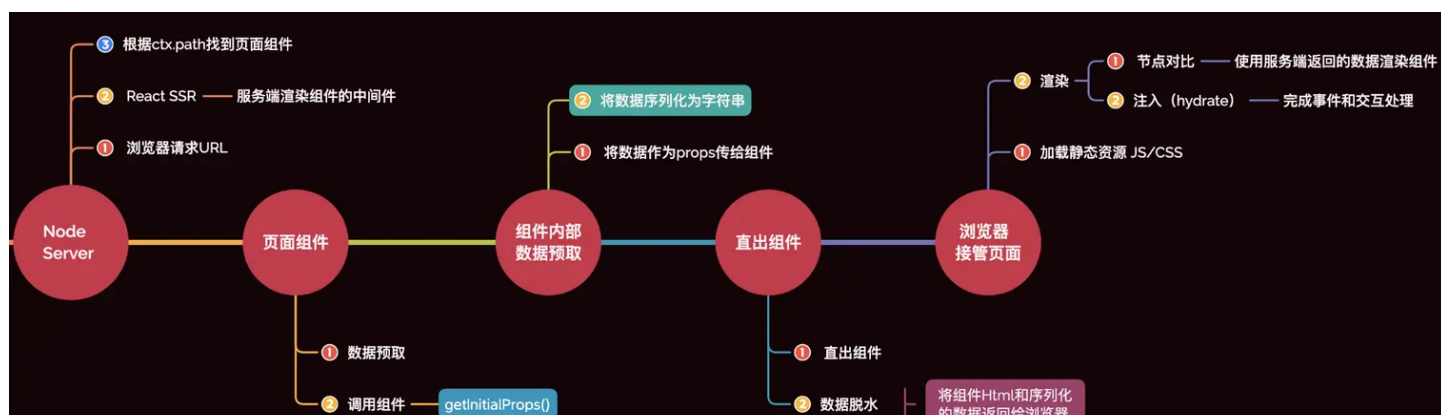
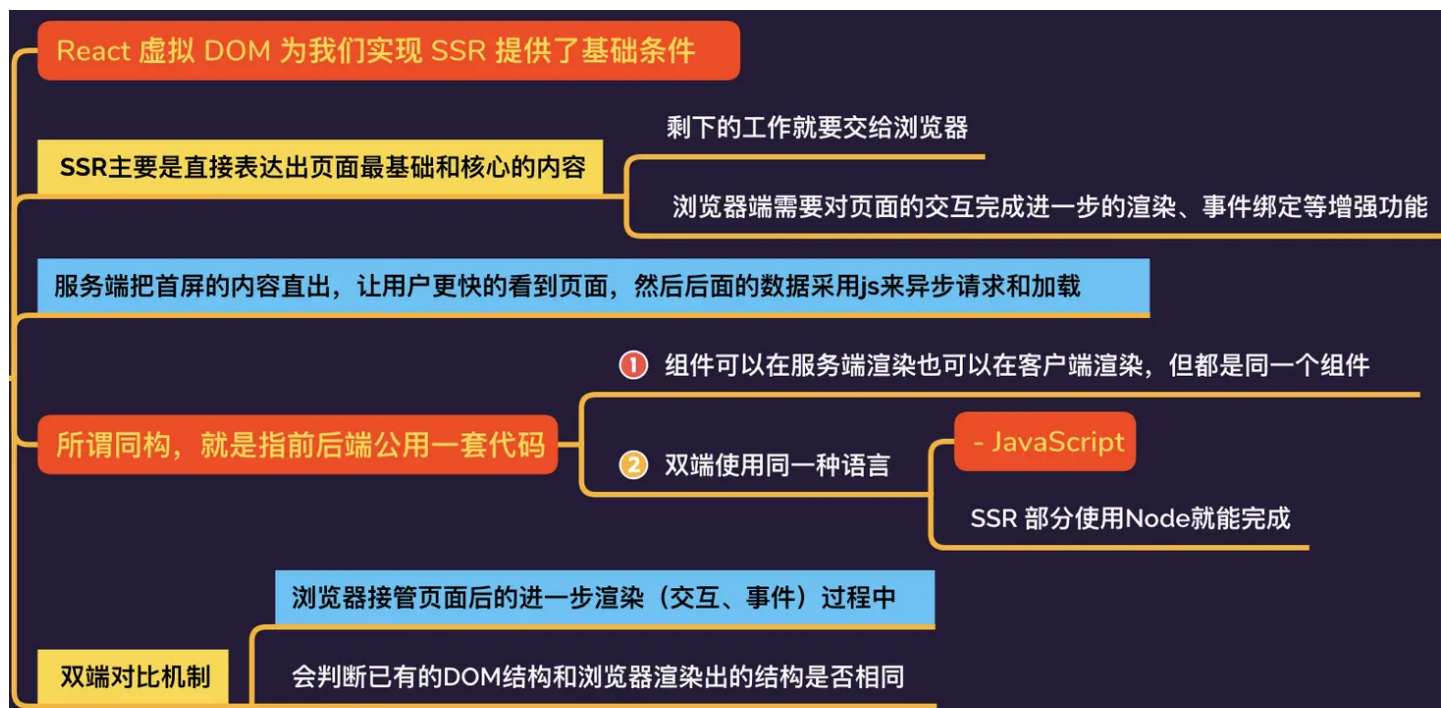
2. 同构

3. 双端对比

VDOM



同构



双端对比





renderToStaticMarkup()

```
1 ReactDOMServer.renderToStaticMarkup(element)
```

仅仅是为了将组件渲染为html字符串，不会带有 `data-react-checksum` 属性

SPR (Serverless Pre-Rendering)

无服务预渲染，这是 `Serverless` 话题之下的一项渲染技术。`SPR` 是指在 `SSR` 架构下通过预渲染与缓存能力，将部分页面转化为静态页面，以避免其在服务器接收到请求的时候频繁被渲染的能力，同时一些框架还支持设置静态资源过期时间，以确保这部分“静态页面”也能有一定的即时性。

SSG (Static Site Generation)

- 它与 `CSR` 一样，只需要**页面托管**，不需要真正编写并部署服务端，页面资源在编译完成部署之前就已经确定；
- 但它又与 `SSR` 一样，属于一种 `Prerender` 预渲染操作，即在用户浏览器得到页面响应之前，页面内容和结构就已经渲染好了。
- 当然形式和特征来看，它更接近 `SSR`。

`SSG` 模式，把原本日益动态化、交互性增强的页面，变成了大部分已经填充好，托管在页面服务 / CDN 上的**静态页面**

NSR (Native Side Rendering)

`Native` 就是客户端，万物皆可**分布式**，可以理解为这就是一种分布式的 `SSR`，不过这里的渲染工作交给了客户端去做而不是远端服务器。在用户即将访问页面的**上级页面预取页面数据**，由**客户端缓存 HTML 结构**，以达到用户真正访问时快速响应的效果。

NSR 见于各种移动端 + `Webview` 的 `Hybrid` 场景，是需要页面与客户端研发协作的一种优化手段。

ESR (Edge Side Rendering)

`Edge` 就是边缘，类比前面的各种 `XSR`，`ESR` 就是将渲染工作交给边缘服务器节点，常见的就是 `CDN` 的边缘节点。这个方案主打的是**边缘节点相比核心服务器与用户的距离优势**，利用了 `CDN` 分级缓存的概念，渲染和内容填充也可以是分级进行并缓存下来的。

`ESR` 之下静态内容与动态内容是分流的，

1. 边缘 `CDN` 节点可以将静态页面内容先响应给用户
2. 然后再自己发起动态内容请求，得到核心服务器响应之后再返回给用户

是在大型网络架构下非常极致的一种优化，但这也就依赖更庞大的技术基建体系了。

ISR (Incremental Site Rendering)

增量式网站渲染，就是对待页面内容小刀切，有**更细的差异化渲染粒度**，能渐进、分层地进行渲染。

常见的选择是：

- 对于重要页面如首屏、访问量较大的直接落地页，进行**预渲染并添加缓存**，保证最佳的访问性能；
- 对于次要页面，则确保有兜底内容可以即时 `fallback`，再将其实时数据的渲染留到 `CSR` 层次完成，同时触发异步缓存更新。

对于“异步缓存更新”，则需要提到一个常见的内容缓存策略：`Stale While Revalidate`，CDN 对于数据请求始终首先响应缓存内容，如果这份内容已经过期，则在响应之后再触发异步更新——这也是对于次要元素或页面的缓存处理方式。

WebComponents

`Web Components` 是一套不同的技术，允许您创建可重用的定制元素并且在您的 web 应用中使用它们

三要素

1. `Custom elements`（自定义元素）：一组 `JavaScript` API，允许您定义 `custom elements` 及其行为，然后可以在您的用户界面中按照需要使用它们。
 - 通过 `class A extends HTMLElement {}` 定义组件，
 - 通过 `window.customElements.define('a-b', A)` 挂载已定义组件。
2. `Shadow DOM`（影子 DOM）：一组 `JavaScript` API，用于将封装的“影子”DOM 树附加到元素（与主文档 DOM 分开呈现）并控制其关联的功能。
 - 通过这种方式，您可以保持元素的功能私有，这样它们就可以被脚本化和样式化，而不用担心与文档的其他部分发生冲突。
 - 使用 `const shadow = this.attachShadow({mode : 'open'})` 在 `WebComponents` 中开启。
3. `HTML templates`（HTML 模板）`slot : template` 可以简化生成 `dom` 元素的操作，不再需要 `createElement` 每一个节点。

虽然 `WebComponents` 有三个要素，但却不是缺一不可的，`WebComponents`

- 借助 `shadow dom` 来实现样式隔离，
 - 借助 `templates` 来简化标签的操作。
-

内部生命周期函数（4个）

1. `connectedCallback`：当 `WebComponents` 第一次被挂在到 `dom` 上是触发的钩子，并且只会触发一次。
 - 类似 `React` 中的 `useEffect(() => {}, [])`，`componentDidMount`。
2. `disconnectedCallback`：当自定义元素与文档 `DOM` 断开连接时被调用。
3. `adoptedCallback`：当自定义元素被移动到新文档时被调用。
4. `attributeChangedCallback`：当自定义元素的被监听属性变化时被调用。

组件通信

传入复杂数据类型

- 传入一个 `JSON` 字符串配饰 `attribute`
 - `JSON.stringify` 配置指定属性
 - 在组件 `attributeChangedCallback` 中判断对应属性，然后用 `JSON.parse()` 获取
- 配置DOM的 `property` 属性
 - `xx.dataSource = [{ name: 'xxx', age: 19 }]`
 - 但是，自定义组件中没有办法监听到这个属性的变化
 - 如果想实现，复杂的结构，不是通过配置，而是在定义组件时候，就确定

状态的双向绑定

```
1 <wl-input id="ipt"
2       :value="data"
3       @change="(e) => { data = e.detail }">
4 </wl-input>
5
6 // js
7 (function () {
8   const template = document.createElement('template')
9   template.innerHTML = `
10   <style>
11     .wl-input {
12
13     }
14   </style>
15   <input type="text" id="wlInput">
16   `
17   class WlInput extends HTMLElement {
18     constructor() {
19       super()
20       const shadow = this.attachShadow({
21         mode: 'closed'
22       })
23       const content = template.content.cloneNode(true)
24       this._input = content.querySelector('#wlInput')
25       this._input.value = this.getAttribute('value')
26       shadow.appendChild(content)
```

```

27     this._input.addEventListener("input", ev => {
28         const target = ev.target;
29         const value = target.value;
30         this.value = value;
31         this.dispatchEvent(
32             new CustomEvent("change", { detail: value })
33         );
34     });
35 }
36 get value() {
37     return this.getAttribute("value");
38 }
39 set value(value) {
40     this.setAttribute("value", value);
41 }
42 }
43 window.customElements.define('wl-input', WlInput)
44 })()

```

监听了这个表单的 `input` 事件，并且在每次触发 `input` 事件的时候触发自定义的 `change` 事件，并且把输入的参数回传。

样式设置

直接给自定义标签添加样式

```

1 <style>
2   wl-input{
3       display: block;
4       margin: 20px;
5       border: 1px solid red;
6   }
7 </style>
8 <wl-input></wl-input>
9 <script src="./index.js"></script>

```

定义元素内部子元素设置样式

分为两种场景：

1. 在主 DOM 使用 JS
2. 在 Custom Elements 构造函数中使用 JS

在主 DOM 使用 JS 给 Shadow DOM 增加 style 标签：

```
1 <script>
2   class WlInput extends HTMLElement {
3     constructor () {
4       super();
5       this.shadow = this.attachShadow({mode: "open"});
6
7       let headerEle = document.createElement("div");
8       headerEle.className = "input-header";
9       headerEle.innerText = "北辰南葵";
10      this.shadow.appendChild(headerEle);
11    }
12  }
13
14  window.customElements.define("wl-input", WlInput);
15
16  // 给 Shadow DOM 增加 style 标签
17  let styleEle = document.createElement("style");
18  styleEle.textContent = `
19    .input-header{
20      padding:10px;
21      background-color: yellow;
22      font-size: 16px;
23      font-weight: bold;
24    }
25  `;
26  document.querySelector("wl-input").shadowRoot.appendChild(styleEle);
27 </script>
```

在 Custom Elements 构造函数中使用 JS 增加 style 标签：

```
1 <script>
2   class WlInput extends HTMLElement {
3     constructor () {
4       super();
5       this.shadow = this.attachShadow({mode: "open"});
6       let styleEle = document.createElement("style");
7       styleEle.textContent = `
8         .input-header{
9           padding:10px;
10          background-color: yellow;
11          font-size: 16px;
12          font-weight: bold;
```

```

13         }
14     `;
15     this.shadow.appendChild(styleEle);
16
17
18     let headerEle = document.createElement("div");
19     headerEle.className = "input-header";
20     headerEle.innerText = "北宸南葵";
21     this.shadow.appendChild(headerEle);
22 }
23 }
24 window.customElements.define("wl-input", WlInput);
25 </script>

```

引入 CSS 文件

使用 JS 创建 link 标签，然后引入 CSS 文件给自定义元素内部的子元素设置样式

```

1 <script>
2     class WlInput extends HTMLElement {
3         constructor () {
4             super();
5             this.shadow = this.attachShadow({mode: "open"});
6             let linkEle = document.createElement("link");
7             linkEle.rel = "stylesheet";
8             linkEle.href = "./my_input.css";
9             this.shadow.appendChild(linkEle);
10
11
12             let headerEle = document.createElement("div");
13             headerEle.className = "input-header";
14             headerEle.innerText = "北宸南葵";
15             this.shadow.appendChild(headerEle);
16         }
17     }
18     window.customElements.define("wl-input", WlInput);
19 </script>

```

样式文件

```

1 .input-header{
2     padding:10px;
3     background-color: yellow;

```

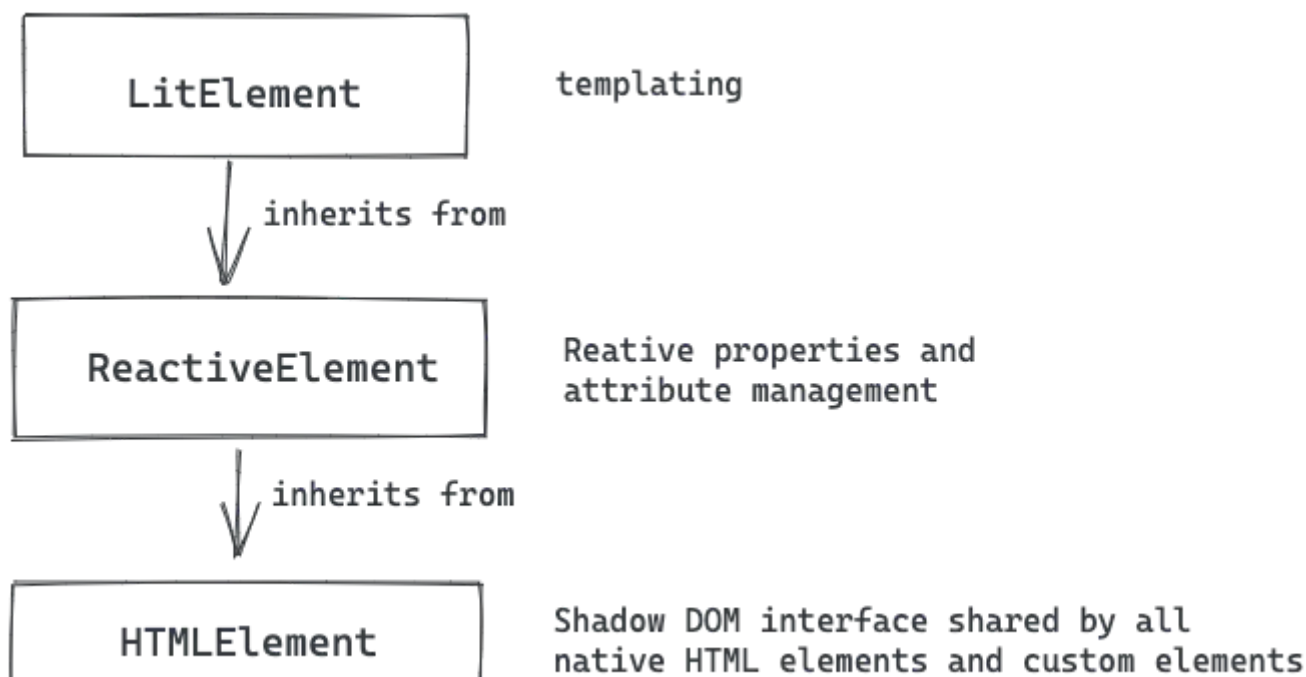
```
4   font-size: 16px;
5   font-weight: bold;
6 }
```

Lit

`Lit` 的核心是一个组件基类，它提供**响应式**、**scoped 样式**和一个小巧、快速且富有表现力的声明性模板系统，且支持 `TypeScript` 类型声明。

Lit 在开发过程中不需要编译或构建，几乎可以在无工具的情况下使用。

我们知道 `HTMLElement` 是浏览器内置的类，`LitElement` 基类则是 `HTMLElement` 的子类，因此 `Lit` 组件继承了所有标准 `HTMLElement` 属性和方法。更具体来说，`LitElement` 继承自 `ReactiveElement`，后者实现了响应式属性，而后者又继承自 `HTMLElement`。



而 `LitElement` 框架则是基于 `HTMLElement` 类二次封装了 `LitElement` 类。

```
1 export class LitButton extends LitElement { /* ... */ }
2 customElements.define('lit-button', LitButton);
```

渲染

组件具有 `render` 方法，该方法被调用以渲染组件的内容。

```
1 export class LitButton extends LitElement {
```

```
2  /* ... */
3
4  render() {
5      // 使用模板字符串，可以包含表达式
6      return html`
7          <div><slot name="btnText"></slot></div>
8      `;
9  }
10 }
11
```

组件的 `render()` 方法返回单个 `TemplateResult` 对象

响应式 properties

DOM 中 `property` 与 `attribute` 的区别：

- `attribute` 是 HTML 标签上的特性，可以理解为标签属性，它的值只能是 `String` 类型，并且会自动添加同名 DOM 属性作为 `property` 的初始值；
- `property` 是 DOM 中的属性，是 `JavaScript` 里的对象，有同名 `attribute` 标签属性的 `property` 属性值的改变也并不会同步引起 `attribute` 标签属性值的改变；

`Lit` 组件接收标签属性 `attribute` 并将其状态存储为 `JavaScript` 的 `class` 字段属性或 `properties`。**响应式 `properties` 是可以在更改时触发响应式更新周期、重新渲染组件以及可选地读取或重新写入 `attribute` 的属性。**每一个 `properties` 属性都可以配置它的选项对象

传入复杂数据类型

对于复杂数据的处理，为什么会存在这个问题，根本原因还是因为 `attribute` 标签属性值只能是 `String` 类型，其他类型需要进行序列化。在 `LitElement` 中，只需要在父组件模板的属性值前使用 `.` 操作符，这样子组件内部 `properties` 就可以正确序列化为目标类型。

优点

`LitElement` 在 `Web Components` 开发方面有着很多比原生的优势，它具有以下特点：

- 简单：在 `Web Components` 标准之上构建，`Lit` 添加了**响应式、声明性模板**和一些周到的功能，**减少了模板文件**。
- 快速：更新速度很快，因为 `Lit` 会跟踪 `UI` 的动态部分，并且只在底层状态发生变化时更新那些部分——无需重建整个虚拟树并将其与 DOM 的当前状态进行比较。
- 轻便：`Lit` 的压缩后大小约为 5 KB，有助于**保持较小的包大小并缩短加载时间**。
- 高扩展性：`lit-html` 基于标记的 `template`，它结合了 ES6 中的模板字符串语法，使得它无需预编译、预处理，就能获得浏览器原生支持，并且扩展能力强。

5. 兼容良好：对浏览器兼容性非常好，对主流浏览器都能有非常好的支持。

npm

嵌套的 node_modules 结构

npm 在早期采用的是嵌套的 node_modules 结构，直接依赖会平铺在 node_modules 下，子依赖嵌套在直接依赖的 node_modules 中。

比如项目依赖了 A 和 C，而 A 和 C 依赖了不同版本的 B@1.0 和 B@2.0，node_modules 结构如下：

```
1 node_modules
2 |— A@1.0.0
3 |   |— node_modules
4 |     |— B@1.0.0
5 |— C@1.0.0
6 |   |— node_modules
7 |     |— B@2.0.0
```

如果 D 也依赖 B@1.0，会生成如下的嵌套结构：

```
1 node_modules
2 |— A@1.0.0
3 |   |— node_modules
4 |     |— B@1.0.0
5 |— C@1.0.0
6 |   |— node_modules
7 |     |— B@2.0.0
8 |— D@1.0.0
9 |   |— node_modules
10 |     |— B@1.0.0
```

可以看到同版本的 B 分别被 A 和 D 安装了两次。

依赖地狱 Dependency Hell

在真实场景下，依赖增多，冗余的包也变多，node_modules 最终会堪比黑洞，很快就能把磁盘占满。而且依赖嵌套的深度也会十分可怕，这个就是依赖地狱。

扁平的 node_modules 结构

为了将嵌套的依赖尽量打平，避免过深的依赖树和包冗余，npm v3 将子依赖提升(hoist)，采用扁平的 node_modules 结构，子依赖会尽量平铺安装在主依赖项所在的目录中。

```
1 node_modules
2 |— A@1.0.0
3 |— B@1.0.0
4 |— C@1.0.0
5     |— node_modules
6         |— B@2.0.0
```

可以看到 A 的子依赖的 B@1.0 不再放在 A 的 node_modules 下了，而是与 A 同层级。

而 C 依赖的 B@2.0 因为版本号原因还是嵌套在 C 的 node_modules 下。

这样不会造成大量包的重复安装，依赖的层级也不会太深，解决了依赖地狱问题，但也形成了新的问题。

幽灵依赖 Phantom dependencies

幽灵依赖是指在 package.json 中未定义的依赖，但项目中依然可以正确地被引用到。

比如上方的示例其实我们只安装了 A 和 C：

```
1 {
2   "dependencies": {
3     "A": "^1.0.0",
4     "C": "^1.0.0"
5   }
6 }
7
```

由于 B 在安装时被提升到了和 A 同样的层级，所以在项目中引用 B 还是能正常工作的。

幽灵依赖是由依赖的声明丢失造成的，如果某天某个版本的 A 依赖不再依赖 B 或者 B 的版本发生了变化，那么就会造成依赖缺失或兼容性问题。

不确定性 Non-Determinism

不确定性是指：同样的 package.json 文件，install 依赖后可能不会得到同样的 node_modules 目录结构。

如果有 `package.json` 变更，本地需要删除 `node_modules` 重新 `install`，否则可能会导致生产环境与开发环境 `node_modules` 结构不同，代码无法正常运行。

依赖分身 Doppelgangers

假设继续再安装依赖 `B@1.0` 的 `D` 模块和依赖 `@B2.0` 的 `E` 模块，此时：

`A` 和 `D` 依赖 `B@1.0` 和 `E` 依赖 `B@2.0` 以下是提升 `B@1.0` 的 `node_modules` 结构：

```
1 node_modules
2 |— A@1.0.0
3 |— B@1.0.0
4 |— D@1.0.0
5 |— C@1.0.0
6 |   |— node_modules
7 |       |— B@2.0.0
8 |— E@1.0.0
9 |   |— node_modules
10 |       |— B@2.0.0
```

可以看到 `B@2.0` 会被安装两次，实际上无论提升 `B@1.0` 还是 `B@2.0`，都会存在重复版本的 `B` 被安装，这两个重复安装的 `B` 就叫 `doppelgangers`。

yarn

`yarn` 也采用扁平化 `node_modules` 结构

提升安装速度

在 `npm` 中安装依赖时，**安装任务是串行的**，会按包顺序逐个执行安装，这意味着它会等待一个包完全安装，然后再继续下一个。

为了加快包安装速度，`yarn` 采用了并行操作，在性能上有显著的提高。而且在缓存机制上，`yarn` 会将**每个包缓存在磁盘上**，在下次安装这个包时，可以脱离网络实现从磁盘离线安装。

lockfile 解决不确定性

`yarn` 更大的贡献是发明了 `yarn.lock`。

在依赖安装时，会根据 `package.json` 生成一份 `yarn.lock` 文件。

`lockfile` 里记录了依赖，以及依赖的子依赖，依赖的版本，获取地址与验证模块完整性的 hash。

即使是不同的安装顺序，相同的依赖关系在任何的环境和容器中，都能得到稳定的

`node_modules` 目录结构，保证了依赖安装的确定性。

所以 `yarn` 在出现时被定义为快速、安全、可靠的依赖管理。而 `npm` 在一年后的 `v5` 才发布了 `package-lock.json`。

与 npm 一样的弊端

`yarn` 依然和 `npm` 一样是扁平化的 `node_modules` 结构，没有解决幽灵依赖和依赖分身问题。

pnpm

内容寻址存储 CAS

与依赖提升和扁平化的 `node_modules` 不同，`pnpm` 引入了另一套依赖管理策略：内容寻址存储。

该策略会将包安装在系统的全局 **store** 中，依赖的每个版本只会在系统中安装一次。

在引用项目 `node_modules` 的依赖时，会通过硬链接与符号链接在全局 `store` 中找到这个文件。为了实现此过程，`node_modules` 下会多出 `.pnpm` 目录，而且是非扁平化结构。

- 硬链接 `Hard link`：硬链接可以理解为源文件的副本，项目里安装的其实是副本，它使得用户可以通过路径引用查找到全局 `store` 中的源文件，而且这个副本根本不占任何空间。同时，`pnpm` 会在全局 `store` 里存储硬链接，不同的项目可以从全局 `store` 寻找到同一个依赖，大大地节省了磁盘空间。
- 符号链接 `Symbolic link`：也叫软连接，可以理解为快捷方式，`pnpm` 可以通过它找到对应磁盘目录下的依赖地址。

由于链接的优势，`pnpm` 的安装速度在大多数场景都比 `npm` 和 `yarn` 快 2 倍，节省的磁盘空间也更多。

yarn Plug' n' Play

`Plug'n'Play`（`Plug'n'Play` = `Plug and Play` = `PnP`，即插即用）。

抛弃 node_modules

无论是 `npm` 还是 `yarn`，都具备缓存的功能，大多数情况下安装依赖时，其实是将缓存中的相关包复制到项目目录中 `node_modules` 里。

而 `yarn PnP` 则不会进行拷贝这一步，而是在项目里维护一张静态映射表 `pnp.cjs`。

npm install 发生了啥

1. 「检查」 `.npmrc` 文件 优先级为：项目级的 `.npmrc` 文件 > 用户级的 `.npmrc` 文件 > 全局级的 `.npmrc` 文件 > `npm` 内置的 `.np`
2. 检查项目中「有无 `lock` 文件」。
3. 无 `lock` 文件：
 - 从 `npm` 远程仓库获取包信息
 - 根据 `package.json` 构建「依赖树」，构建过程：
 - 构建依赖树时，不管其是直接依赖还是子依赖的依赖，优先将其「放置在 `node_modules` 根目录」。
 - 当遇到「相同模块」时，判断已放置在依赖树的模块版本是否符合新模块的版本范围，如果符合则跳过，不符合则在当前模块的 `node_modu`
 - 注意这一步只是确定逻辑上的依赖树，并非真正的安装，后面会根据这个依赖结构去下载或拿到缓存中的依赖包
 - 在「缓存」中依次查找依赖树中的每个包
 - 「不存在缓存」：
 - 从 `npm` 远程仓库下载包
 - 校验包的完整性
 - 校验不通过：
 - 重新下载
 - 校验通过：
 - 将下载的包复制到 `npm` 缓存目录
 - 将下载的包按照依赖结构解压到 `node_modules`
 - 「存在缓存」：将缓存按照依赖结构解压到 `node_modules`
 - 将包解压到 `node_modules`
 - 生成 `lock` 文件
4. 有 `lock` 文件：
 - 检查 `package.json` 中的依赖版本是否和 `package-lock.json` 中的依赖有冲突。
 - 如果「没有冲突」，直接跳过获取包信息、构建依赖树过程，开始在缓存中查找包信息，后续过程相同

使用 history 模式的前端路由时静态资源服务器配置详解

我们一般都是打包以后放在静态资源服务器中的，我们访问诸如 `example.com/rootpath/` 这种形式的资源没问题，是因为，`index.html` 文件是真实的存在于 `rootpath` 文件夹中的，可以找到的，返回给前端的。

但是如果访问子路由 `example.com/rootpath/login` 进行登录操作，但是 `login/index.html` 文件并非真实存在的文件，其实我们需要的文件还是 `rootpath` 目录中的 `index.html`。

再者，如果我们需要 `js` 文件，比如登陆的时候请求的地址是 `example.com/rootpath/login/js/dist.js` 其实我们想要的文件，还是 `rootpath/js/` 目录中的 `dist.js` 文件而已。

前端路由其实是一种假象，只是用来蒙蔽使用者而已的，无论用什么路由，访问的都是同一套静态资源。

之所以展示的内容不同，只是因为代码里，根据不同的路由，对要显示的视图做了处理而已。

比如

- 要找 `example.com/rootpath/login` 静态资源服务器找不到，那就返回 `example.com/rootpath/` 内容；
- 要找 `example.com/rootpath/login/css/style.css` 找不到，那就照着 `example.com/rootpath/css/style.css` 这个路径去找。

总之就是，请求的是子目录，找不到，那就返回根目录一级对应的资源文件就好了。

在 nginx 中使用

如果你打包以后的前端静态资源文件，想要仍在 `nginx` 中使用，那首先将你打包好的静态资源目录扔进 `www` 目录，比如你打包好的资源的目录叫 `rootpath`，那么直接将 `rootpath` 整个目录丢进 `www` 目录即可。

然后打开我们的 `nginx` 配置文件 `nginx.conf`，插入以下配置：

```
1 location /rootpath/ {
2     root    html;
3     index  index.html index.htm;
4     try_files $uri $uri/ /rootpath/index.html;
5 }
```

1. `root` 的作用

- 就是指定一个根目录。默认的是 `html` 目录

2. `try_files`

- 关键点1：按指定的 `file` 顺序查找存在的文件，并使用第一个找到的文件进行请求处理
- 关键点2：查找路径是按照给定的 `root` 或 `alias` 为根路径来查找的
- 关键点3：如果给出的 `file` 都没有匹配到，则重新请求最后一个参数给定的 `uri`，就是新的 `location` 匹配

webpack 优化

时间方向(8个)

1. 开发环境 - `EvalSourceMapDevToolPlugin` 排除第三方模块
 - `devtool:false`
 - `EvalSourceMapDevToolPlugin`, 通过传入 `module: true` 和 `column:false`, 达到和预设 `eval-cheap-module-source-map` 一样的质量
2. 缩小 `loader` 的搜索范围: `test`、`include`、`exclude`
3. `Module.noParse`
 - `noParse: /jquery|lodash/`,
4. `TypeScript` 编译优化
5. `Resolve.modules` 指定查找模块的目录范围
6. `Resolve.alias`
7. `Resolve.extensions` 指定查找模块的文件类型范围
8. `HappyPack`

资源大小 (9个)

1. 按需引入类库模块 (工具类库)
 - 使用 `babel-plugin-import` 对其处理
2. 使用 `externals` 优化 `cdn` 静态资源
 - **CSS抽离+剔除无用样式** - `MiniCssExtractPluginPurgeCSS`
 - **CSS压缩** `CssMinimizerWebpackPlugin`
1. `TreeShaking`
 - CSS 方向 - `glob-all` `purify-css` `purifycss-webpack`
 - JS方向 - `babel-loader` 版本问题
 - `Code SpiltoptimizationsplitChunkschunks:all`
1. 魔法注释 - `webpackChunkName: 'xxx'`
 - `Scope HoistingoptimizationconcatenateModules:true`
 - 普通打包只是将一个模块最终放入一个单独的函数中, 如果模块很多, 就意味着在输出结果中会有很多的模块函数。 `concatenateModules` 配置的作用, 尽可能将所有模块合并到一起输出到一个函数中, 既提升了运行效率, 又减少了代码的体积。
 - **图片压缩** `image-webpack-loader` 只要在 `file-loader` 之后加入 `image-webpack-loader` 即可

共同方案

Redux内部实现

createStore

```
1 function createStore(  
2   reducer,  
3   preloadedState,  
4   enhancer  
5 ){  
6   let state;  
7  
8   // 用于存放被 subscribe 订阅的函数 (监听函数)  
9   let listeners = [];  
10  
11   // getState 是一个很简单的函数  
12   const getState = () => state;  
13  
14   return {  
15     dispatch,  
16     getState,  
17     subscribe,  
18     replaceReducer  
19   }  
20 }
```

dispatch

```
1 function dispatch(action) {  
2   // 通过 reducer 返回新的 state  
3   // 这个 reducer 就是 createStore 函数的第一个参数  
4   state = reducer(state, action);  
5   // 每一次状态更新后, 都需要调用 listeners 数组中的每一个监听函数  
6   listeners.forEach(listener => listener());  
7   return action;    // 返回 action  
8 }
```

subscribe

```

1 function subscribe(listener){
2   listeners.push(listener);
3   // 函数取消订阅函数
4   return () => {
5     listeners = listeners.filter(fn => fn !== listener);
6   }
7 }

```

combineReducers

```

1 function combineReducers(reducers){
2   return (state = {},action) => {
3     // 返回的是一个对象， reducer 就是返回的对象
4     return Object.keys(reducers).reduce(
5       (accum,currentKey) => {
6         accum[currentKey] = reducers[currentKey]
7         (state[currentKey],action);
8         return accum;
9       },{} // accum 初始值是空对象
10    );
11  }

```

applyMiddleware

```

1 function applyMiddleware(...middlewares){
2   return function(createStore){
3     return function(reducer,initialState){
4       var store = createStore(reducer,initialState);
5       var dispatch = store.dispatch;
6       var chain = [];
7
8       var middlewareAPI = {
9         getState: store.getState,
10        dispatch: (action) => dispatch(action)
11      };
12
13      chain = middlewares.map(
14        middleware => middleware(middlewareAPI)
15      );
16
17      dispatch = compose(...chain)(store.dispatch);

```

```
18     return { ...store, dispatch };
19   }
20 }
21 }
```

`applyMiddleware` 函数是一个三级柯里化函数

Vue和 React的区别

共同点

1. 数据驱动视图
2. 组件化
3. 都使用 `Virtual DOM`

不同点

1. 核心思想
 - `Vue` 灵活易用的渐进式框架，进行**数据拦截/代理**，它对侦测数据的变化更敏感、更精确
 - `React` 推崇**函数式编程**（纯组件），**数据不可变以及单向数据流**
2. 组件写法差异
 - `React` 推荐的做法是 `JSX + inline style`，也就是把 `HTML` 和 `CSS` 全都写进 JavaScript 中，即 `all in js`；
 - `Vue` 推荐的做法是 `template` 的**单文件组件格式**即 `html`，`css`，`JS` 写在同一个文件
3. `diff` 算法不同
 - 两者流程思路是类似的：不同的组件产生不同的 DOM 结构。**当type不不同时，对应DOM操作就是直接销毁老的DOM，创建新的DOM。同一层次的一组子节点，可以通过唯一的 key 区分。**
 - `Vue-Diff` 算法采用了**双端比较的算法**，同时从新旧 `children` 的两端开始进行比较，借助 `key` 值找到可复用的节点，再进行相关操作。相比 `React` 的 `Diff` 算法，同样情况下可以减少移动节点次数，减少不必要的性能损耗，更加的优雅。
4. 响应式原理不同
 - `Vue` 依赖收集，自动优化，**数据可变**，当数据改变时，自动找到引用组件重新渲染
 - `React` 基于**状态机**，手动优化，**数据不可变**，需要 `setState` 驱动新的 `state` 替换老的 `state`。当数据改变时，以组件为根目录，默认全部重新渲染。

Webpack有哪些常用的loader和plugin

Webpack Loader vs Plugin

- `loader` 是**文件加载器**，能够加载资源文件，并对这些文件进行一些处理，诸如编译、压缩等，最终一起打包到指定的文件中
- `plugin` 赋予了 `webpack` 各种灵活的功能，例如打包优化、资源管理、环境变量注入等，目的是**解决 loader 无法实现的其他事**
- `loader` 运行在**打包文件之前**
- `plugins` 在整个编译周期都起作用

常用loader

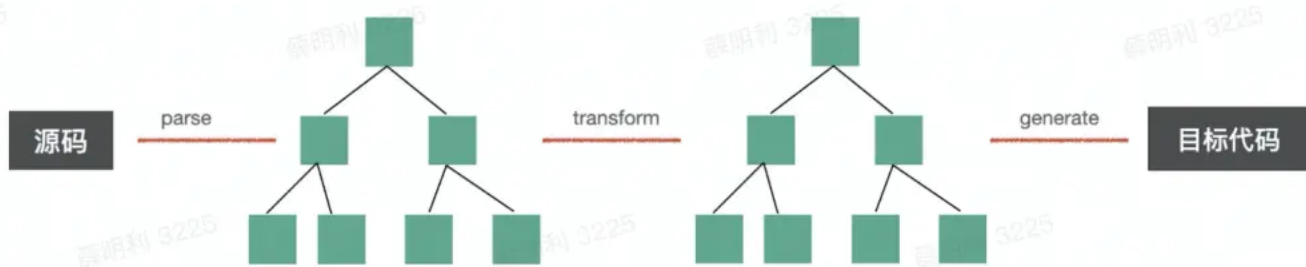
- 样式: `style-loader`、`css-loader`、`less-loader`、`sass-loader`、`MiniCssExtractPluginPurgeCSSCssMinimizerWebpackPlugin`
- js: `babel-loader` / `ts-loader`
- 图片: `url-loader` (`limit`)、`file-loader`、`image-webpack-loader`
- 代码校验: `eslint-loader`

常用plugin

1. `HtmlWebpackPlugin` : 会在打包结束之后自动创建一个 `index.html` , 并将打包好的JS自动引入到这个文件中
2. `MiniCssExtractPlugin`
3. `IgnorePlugin` : 用于**忽略第三方包**指定目录，让指定目录不被打包进去
4. `terser-webpack-plugin` : 压缩js代码
5. `SplitChunksPlugin` : `Code-Splitting` 实现的底层就是通过Split-Chunks-Plugin实现的，其作用就是代码分割。

Babel

`Babel` 是一个 `JavaScript` 编译器！



Babel 的作用就是将**源码**转换为**目标代码**

Babel的作用

主要用于将采用 `ECMAScript 2015+` 语法编写的代码转换为 `es5` 语法，让开发者无视用户浏览器的差异性，并且能够用**新的 JS 语法及特性**进行开发。除此之外，Babel 能够转换 `JSX` 语法，并且能够支持 `TypeScript` 转换为 `JavaScript`。

总结一下：Babel 的作用如下

1. 语法转换
2. 通过 `Polyfill` 方式在目标环境中**添加缺失的特性**
3. 源码转换

原理

Babel 的运行原理可以通过以下这张图来概括。整体来看，可以分为三个过程，分别是：

1. 解析，
 - a. 词法解析
 - b. 语法解析
2. 转换，
3. 生成。

Babel7 的使用

Babel 支持多种形式的配置文件，根据使用场景不同可以选择不同的配置文件。

- 如果配置中需要**书写 js 逻辑**，可以选择 `babel.config.js` 或者 `.babelrc.js`；
- 如果只是需要一个简单的 `key-value` 配置，那么可以选择 `.babelrc`，甚至可以直接在 `package.json` 中配置。

所有 Babel 的包都发布在 `npm` 上，并且名称以 `@babel` 为前缀（自从版本 7.0 之后），接下来，我们一起看下 `@babel/core` 和 `@babel/cli` 这两个 `npm` 包。

- `@babel/core` 核心库，封装了 `Babel` 的核心能力
- `@babel/cli` 命令行工具，提供了 `babel` 这个命令

`Babel` 构建在插件之上的。默认情况下，`Babel` 不做任何处理，需要借助插件来完成语法的解析，转换，输出。

插件的**配置形式**常见有两种，分别是

1. 字符串格式
2. 数组格式，并且可以**传递参数**

如果插件名称为 `@babel/plugin-XXX`，可以使用简写成 `@babel/XXX`，

- 例如 `@babel/plugin-transform-arrow-functions` 便可以简写成 `@babel/transform-arrow-functions`。

插件的执行顺序是从前往后。

```
1 // .babelrc
2 /*
3 * 以下三个插件的执行顺序是：
4   @babel/proposal-class-properties ->
5   @babel/syntax-dynamic-import ->
6   @babel/plugin-transform-arrow-functions
7 */
8 {
9   "plugins": [
10     // 同 "@babel/plugin-proposal-class-properties"
11     "@babel/proposal-class-properties",
12     // 同 ["@babel/plugin-syntax-dynamic-import"]
13     ["@babel/syntax-dynamic-import"],
14     [
15       "@babel/plugin-transform-arrow-functions",
16       {
17         "loose": true
18       }
19     ]
20   ]
21 }
22
23
```

预设

预设是一组插件的集合。

与插件类似，预设的配置形式也是**字符串**和**数组**两种，预设也可以将 `@babel/preset-XXX` 简写为 `@babel/XXX`。

预设的执行顺序是从后往前，并且**插件在预设之前执行**。

我们常见的预设有以下几种：

- `@babel/preset-env`：可以**无视浏览器环境的差异**而尽情地使用 ES6+ 新语法和新特性；
 - 注：语法和特性不是一回事，语法上的迭代是让我们书写代码更加简单和方便，如展开运算符、类，结构等，因此这些语法称为语法糖；特性上的迭代是为了扩展语言的能力，如 `Map`、`Promise` 等，
 - 事实上，`Babel` 对新语法和新特性的处理也是不一样的，对于新语法，`Babel` 通过插件直接转换，而对于新特性，`Babel` 还需要借助 `polyfill` 来处理和转换。
- `@babel/preset-react`：可以书写 `JSX` 语法，将 `JSX` 语法转换为 `JS` 语法；
- `@babel/preset-typescript`：可以使用 `TypeScript` 编写程序，将 `TS` 转换为 `JS`；
 - 注：该预设只是将 `TS` 转为 `JS`，不做任何类型检查
- `@babel/preset-flow`：可以使用 `Flow` 来控制类型，将 `Flow` 转换为 `JS`；

```
1 // .babelrc
2 /*
3  * 预设的执行顺序为：
4   @babel/preset-react ->
5   @babel/preset-typescript ->
6   @babel/preset-env
7  */
8 {
9   "presets": [
10     [
11       "@babel/preset-env",
12       {
13         "useBuiltIns": "usage",
14         "corejs": {
15           "version": 3,
16           "proposals": true // 使用尚在提议阶段特性的 polyfill
17         }
18       ]
19     ],
20     "@babel/preset-typescript",
21     // 同 @babel/preset-react
22     "@babel/react"
23   ]
24 }
```

对于 `@babel/preset-env`，我们通常需要设置目标浏览器环境，可以在根目录下的 `.browserslistrc` 文件中设置，也可以在该预设的参数选项中通过 `targets` (优先级最高) 或者在 `package.json` 中通过 `browserslist` 设置。

如果我们不设置的话，该预设默认会将所有的 ES6+ 的**新语法**全部做转换，否则，该预设只会对目标浏览器环境**不兼容的新语法**做转换。

推荐设置目标浏览器环境，这样在中大型项目中可以明显缩小编译后的代码体积，因为**有些新语法的转换需要引入一些额外定义的 helper 函数的**，比如 `class`。

`.babelrc`

```
1 {
2   "presets": [
3     [
4       "@babel/preset-env",
5       {
6         "targets": "> 0.25%, not dead"
7       }
8     ]
9   ]
10 }
```

`.browserslistrc`

```
1
2 > 0.25%
3 not dead
```

对于新特性，`@babel/preset-env` 也是能转换的。但是需要通过 `useBuiltIns` 这个参数选项实现，值需要设置为 `usage`，这样的话，只会转换我们使用到的**新语法和新特性**，能够有效减小编译后的包体积，并且还要设置 `corejs: { version: 3, proposals }` 选项，因为转换新特性需要用到 `polyfill`，而 `corejs` 就是一个 `polyfill` 包。如果不显示指定 `corejs` 的版本的话，默认使用的是 `version 2`，而 `version 2` 已经停更，诸如一些更新的特性的 `polyfill` 只会更行与 `version 3` 里，如 `Array.prototype.flat()`。

```
1 // .babelrc
2 "presets": [
3   [
4     "@babel/preset-env",
5     {
```

```

6         "useBuiltIns": "usage",
7         "corejs": {
8             "version": 3,
9             "proposals": true // 使用尚在提议阶段特性的 polyfill
10        }
11    }
12 ]
13 ]
14

```

虽然 `@babel/env` 可以帮我们做新语法和新特性的**按需转换**，但是依然存在 2 个问题：

1. 从 `corejs` 引入的 `polyfill` 是**全局范围**的，不是模块作用域返回的，可能存在污染全局变量的风险；
2. 对于某些新语法，如 `class`，会在编译后的文件中注入很多 `helper` 函数声明，而不是从某个地方 `require` 进来的函数引用，从而增大编译后的包体积；

runtime

`runtime` 是 `babel7` 提出来的概念，旨在解决如上提出的性能问题的。

实践一下 `@babel/plugin-transform-runtime` 插件配合 `@babel/preset-env` 使用

```

1 npm install --save-dev @babel/plugin-transform-runtime
2 // @babel/runtime 是要安装到生产依赖的，因为新特性的编译需要从这个包里引用 polyfill
3 // 它就是一个封装了 corejs 的 polyfill 包
4 npm install --save @babel/runtime

```

```

1 // .babelrc
2 {
3   "presets": [
4     "@babel/env"
5   ],
6   "plugins": [
7     [
8       "@babel/plugin-transform-runtime",{
9         "corejs": 3
10      }
11    ]
12  ],
13 }

```

编译后，可以明显看到，

- 引入的 `polyfill` 不再是全局范围内的了，而是模块作用域范围内的；
- 并且不再是往编译文件中直接注入 `helper` 函数了，而是通过引用的方式，

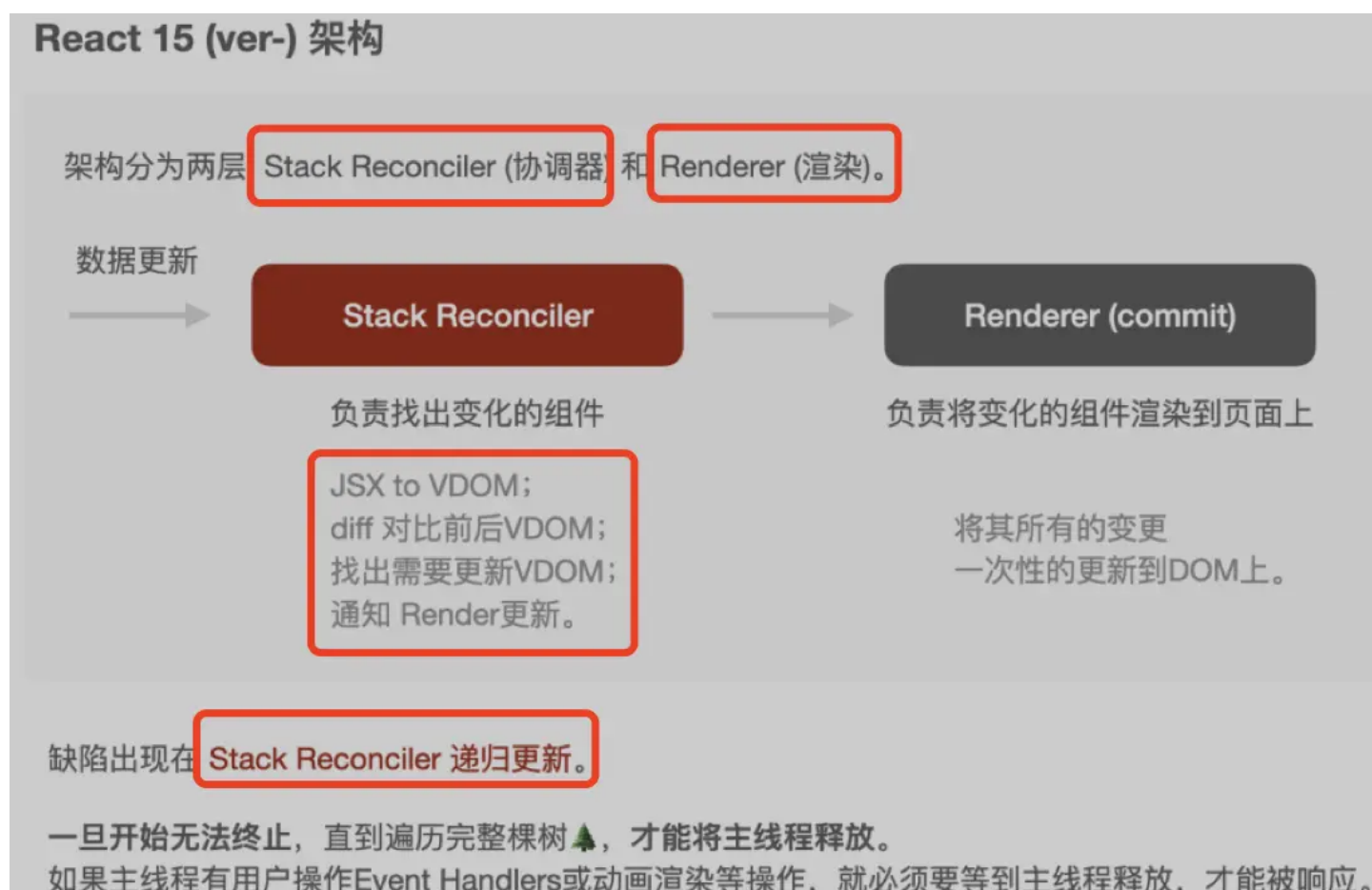
既解决了全局变量污染的问题，又减小了编译后包的体积

Fiber 实现时间切片的原理

React15 架构缺点

React16之前 的版本比对更新虚拟DOM的过程是采用循环递归方式来实现的，这种比对方式有一个问题，就是一旦任务开始进行就**无法中断**，如果应用中数组数量庞大，主线程被长期占用，直到整颗虚拟DOM树比对更新完成之后主线程才被释放，主线程才能执行其他任务，这就会导致**一些用户交互或动画等任务无法立即得到执行，页面就会产生卡顿，非常的影响用户体验。**

主要原因就是递归无法中断，执行重的任务耗时较长，`javascript` 又是单线程的，无法同时执行其他任务，导致任务延迟页面卡顿用户体验差。



Fiber架构

界面通过 `vdom` 描述，但是不是直接手写 `vdom`，而是 `jsx` 编译产生的 `render` function 之后以后生成的。这样就可以加上 `state`、`props` 和一些**动态逻辑**，动态产生 `vdom`。

`vdom` 生成之后不再是直接渲染，而是先转成 `fiber`，这个 `vdom` 转 `fiber` 的过程叫做 `reconcile`。

`fiber` 是一个链表结构，可以打断，这样就可以通过 `requestIdleCallback` 来空闲调度 `reconcile`，这样不断的循环，直到处理完所有的 `vdom` 转 `fiber` 的 `reconcile`，就开始 `commit`，也就是更新到 `dom`。

`reconcile` 的过程会提前创建好 `dom`，还会**标记出增删改**，那么 `commit` 阶段就很快了。

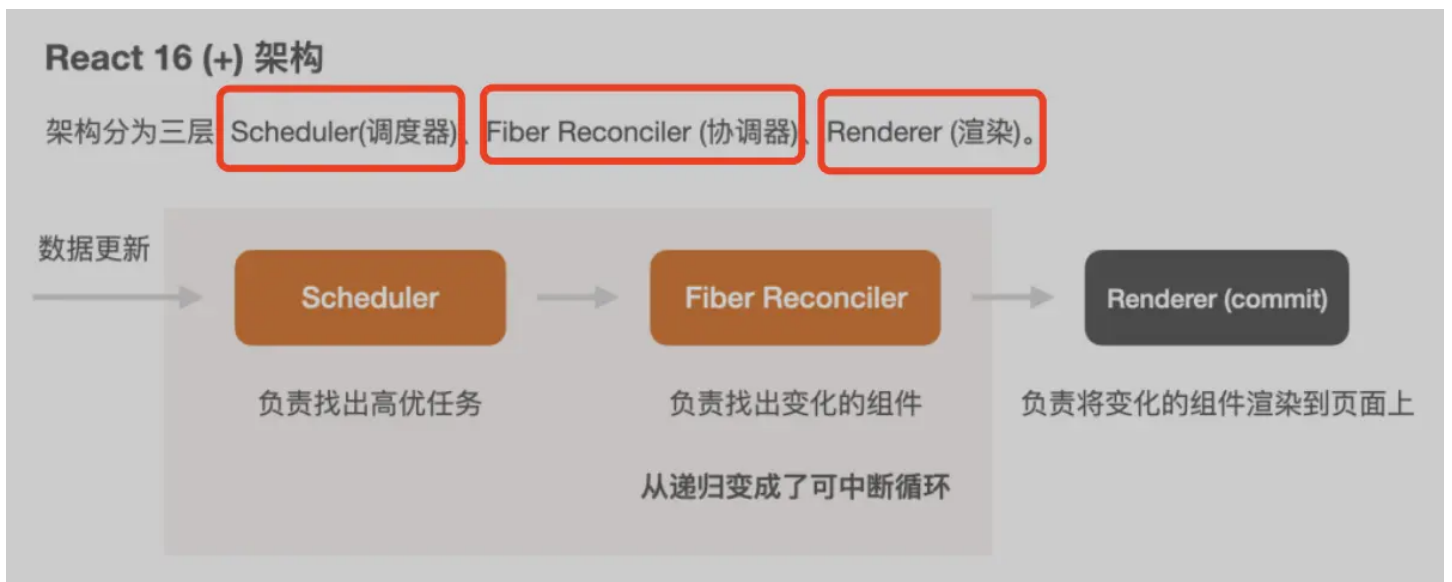
从之前递归渲染时做 `diff` 来确定增删改以及创建 `dom`，提前到了可打断的 `reconcile` 阶段，让 `commit` 变得非常快，这就是 `fiber` 架构的目的和意义。

并发&调度(Concurrency & Scheduler)

- `Concurrency` 并发: **有能力优先处理更高优事务**，同时对正在执行的中途任务可暂存，待高优完成后，再去执行。
- `Scheduler` 协调调度: 暂存未执行任务，等待时机成熟后，再去**安排执行剩下未完成任务**。

考虑到可中断渲染，并可重回构造。 `React` 自行实现了一套体系叫做 `React fiber` 架构。

`React Fiber` 核心: 自行实现 虚拟栈帧。



`schedule` 就是通过**空闲调度**每个 `fiber` 节点的 `reconcile` (`vdom` 转 `fiber`)，全部 `reconcile` 完了就执行 `commit`。

`Fiber` 的数据结构有三层信息: (**采用链表结构**)

1. 实例属性
 - 该 `Fiber` 的基本信息，例如组件类型等。
2. 构建属性
 - 构建属性 (`return`、`child`、`sibling`)

3. 工作属性

- 数据的变更会导致UI层的变更
- 为了减少对 DOM 的直接操作，通过 Reconcile 进行 diff 查找，并将需要变更节点，打上标签，变更路径保留在 effectList 里
- 待变更内容要有 Scheduler 优先级处理
- 涉及到 diff 等查找操作，是需要有个高效手段来处理前后变化，即双缓存机制。

链表结构即可支持随时随时中断的诉求

Scheduler 运行核心点

1. 有个任务队列 queue，该队列存放可中断的任务。
2. workLoop 对队列里取第一个任务 currentTask，进入循环开始执行。
 - 当该任务没有时间或需要中断(渲染任务或其他高优任务插入等)，则让出主线程。
3. requestAnimationFrame 计算一帧的空余时间；
4. 使用 new MessageChannel () 执行宏任务；

devServer进行跨域处理

```
1 module.exports = {
2   devServer: {
3     /* 运行代码的目录 */
4     contentBase: resolve(__dirname, "dist"),
5     /* 监视 contentBase 目录下的所有文件,一旦文件发生变化就会 reload (重载+刷新浏览器)*/
6     watchContentBase: true,
7     /* 监视文件时 配合 watchContentBase */
8     watchOptions: {
9       /* 忽略掉的文件(不参与监视的文件) */
10      ignored: /node_modules/
11    },
12    /* 启动gzip压缩 */
13    compress: true,
14    /* 运行服务时自动打开服务器 */
15    open: true,
16    /* 启动HMR热更新 */
17    hot: true,
18    /* 启动的端口号 */
19    port: 5000,
20    /* 启动的IP地址或域名 */
```

```
21     host: "localhost",
22     /* 关闭服务器启动日志 */
23     clientLogLevel: "none",
24     /* 除了一些启动的基本信息,其他内容都不要打印 */
25     quiet: true,
26     /* 如果出错不要全屏提示 */
27     overlay: false,
28     /* 服务器代理 --> 解决开发环境跨域问题 */
29     proxy: {
30         /* 一旦devServer(port:5000)服务器接收到 ^/api/xxx 的请求,就会把请求转发
    到另外一个服务器(target)上 */
31         "/api": {
32             target: "http://localhost:3000",
33             /* 路径重写(代理时发送到target的请求去掉/api前缀) */
34             pathRewrite: {
35                 "^/api": ""
36             }
37         }
38     },
39 },
40 }
41
```

React 实现原理

React-Hook为什么不能放到条件语句中

每一次渲染都是完全独立的。

```
<App />
```

每次渲染具有独立的状态值（每次渲染都是完全独立的）。也就是说，每个函数中的 `state` 变量只是一个简单的常量，每次渲染时从钩子中获取到的常量，并没有附着数据绑定之类的神奇魔法。

这也就是老生常谈的 `Capture Value` 特性。可以看下面这段经典的计数器代码

```
1 function Counter() {
2   const [count, setCount] = useState(0);
3
4   function handleAlertClick() {
5     setTimeout(() => {
6       alert('You clicked on: ' + count);
7     }, 3000);
8   }
9
10  return (
11    <div>
12      <p>You clicked {count} times</p>
13      <button onClick={() => setCount(count + 1)}>
14        Click me
15      </button>
16      <button onClick={handleAlertClick}>
17        Show alert
18      </button>
19    </div>
20  );
21 }
22
```

按如下步骤操作：

- 1) 点击 `Click me` 按钮，把数字增加到 3；
- 2) 点击 `Show alert` 按钮；
- 3) 在 `setTimeout` 触发之前点击 `Click me`，把数字增加到 5。

结果是 `Alert` 显示 3！

来简单解释一下：

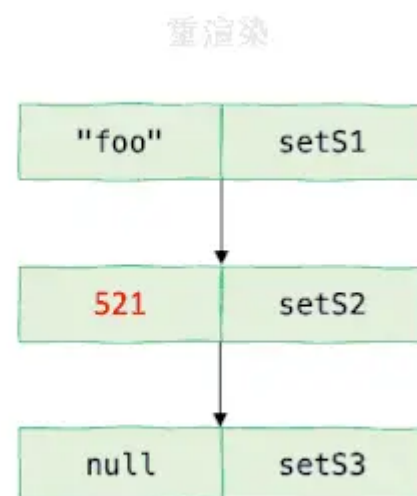
- 每次**渲染相互独立**，因此每次渲染时组件中的状态、事件处理函数等等都是独立的，或者说只属于所在的那一次渲染
- 我们在 `count` 为 3 的时候触发了 `handleAlertClick` 函数，这个函数所记住的 `count` 也为 3
- 三秒种后，刚才函数的 `setTimeout` 结束，输出当时记住的结果：3

深入useState本质

当组件初次渲染（挂载）时

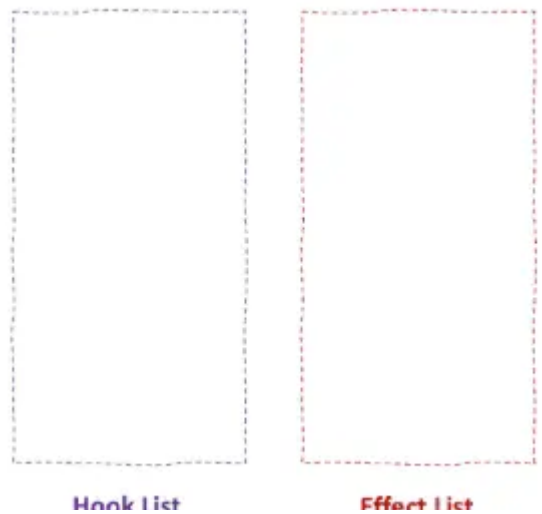
1. 在初次渲染时，我们通过 `useState` 定义了多个状态；
2. 每调用一次 `useState`，都会在组件之外生成一条 **Hook 记录**，同时包括状态值（用 `useState` 给定的初始值初始化）和修改状态的 `Setter` 函数；
3. 多次调用 `useState` 生成的 **Hook 记录形成了一条链表**；
4. 触发 `onClick` 回调函数，调用 `setS2` 函数修改 `s2` 的状态，不仅修改了 **Hook 记录** 中的状态值，还即将**触发重渲染**。

组件重渲染时



在初次渲染结束之后、重渲染之前，**Hook 记录链表**依然存在。当我们**逐个调用** `useState` 的时候，`useState` 便**返回了 Hook 链表中存储的状态**，以及修改状态的 `Setter`。

深入useEffect本质



注意其中一些细节：

- `useState` 和 `useEffect` 在每次调用时都被添加到 `Hook` 链表中；
- `useEffect` 还会额外地在一个队列中添加一个等待执行的 `Effect` 函数；
- 在**渲染完成后**，依次调用 `Effect` 队列中的每一个 `Effect` 函数。

`React` 官方文档 `Rules of Hooks` 中强调过一点：

Only call hooks at the top level. 只在最顶层使用 Hook。

具体地说，不要在循环、嵌套、条件语句中使用 `Hook` ——

因为这些动态的语句很有可能会导致每次执行组件函数时调用 `Hook` 的顺序不能完全一致，导致 `Hook` 链表记录的数据失效。

自定义Hook实现原理

组件初次渲染



在 `App` 组件中调用了 `useCustomHook` 钩子。可以看到，即便我们切换到了自定义 Hook 中，Hook 链表的生成依旧没有改变。

组件重新渲染

即便代码的执行进入到自定义 Hook 中，依然可以从 Hook 链表中读取到相应的数据，这个”配对“的过程总能成功。

而 `Rules of Hook`。它规定只有在两个地方能够使用 React Hook：

1. React 函数组件
2. 自定义 Hook

第一点毋庸置疑，第二点通过刚才的两个动画你也可以轻松的得出一个结论：

自定义 Hook 本质上只是把调用内置 Hook 的过程封装成一个个可以复用的函数，并不影响 Hook 链表的生成和读取。

useCallback

依赖数组在判断元素是否发生改变时使用了 `Object.is` 进行比较，因此当 `deps` 中某一元素为非原始类型时（例如函数、对象等），每次渲染都会发生改变，从而每次都会触发 `Effect`，失去了 `deps` 本身的意义。

Effect 无限循环

来看一下这段”永不停止“的计数器：

```
1 function EndlessCounter() {
2   const [count, setCount] = useState(0);
3
4   useEffect(() => {
5     setTimeout(() => setCount(count + 1), 1000);
6   });
7
8   return (
9     <div className="App">
10       <h1>{count}</h1>
11     </div>
12   );
13 }
```

如果你去运行这段代码，会发现数字永远在增长。我们来通过一段动画来演示一下这个”无限循环“到底是怎么回事：

组件陷入了：**渲染 => 触发 Effect => 修改状态 => 触发重渲染**的无限循环

关于记忆化缓存（Memoization）

Memoization，一般称为记忆化缓存（或者“记忆”），它背后的思想很简单：假如我们有一个计算量很大的纯函数（给定相同的输入，一定会得到相同的输出），那么我们在第一次遇到特定输入的时候，把它的输出结果“记”（缓存）下来，那么下次碰到同样的输出，只需要从缓存里面拿出来直接返回就可以了，省去了计算的过程！

记忆化缓存（Memoization）的两个使用场景：

1. 通过缓存计算结果，节省费时的计算
2. 保证相同输入下**返回值的引用相等**

useCallback使用方法和原理解析

为了解决函数在多次渲染中的**引用相等**（Referential Equality）问题，**React** 引入了一个重要的 Hook —— **useCallback**。官方文档介绍的使用方法如下：

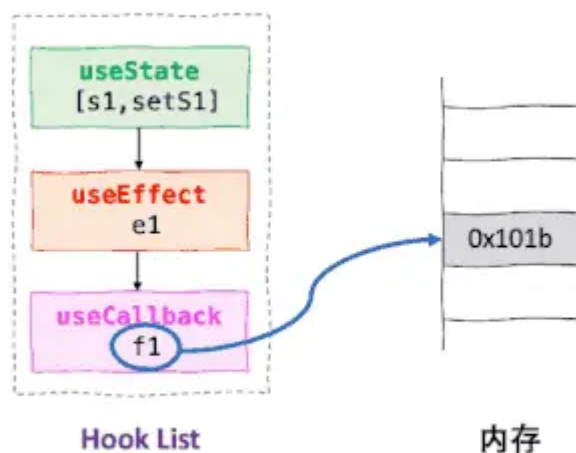
```
1 const memoizedCallback = useCallback(callback, deps);
```

第一个参数 **callback** 就是需要记忆的函数，第二个参数是 **deps** 参数，同样也是一个**依赖数组**。在 **Memoization** 的上下文中，这个 **deps** 的作用相当于缓存中的键（Key），如果**键没有改变**，那么就**直接返回缓存中的函数**，并且**确保是引用相同的函数**。

组件初次渲染(deps 为空数组的情况)

调用 `useCallback` 也是追加到 `Hook` 链表上，不过这里着重强调了这个函数 `f1` 所指向的内存位置，从而明确告诉我们：这个 `f1` 始终是指向同一个函数。然后返回的 `onClick` 则是指向 `Hook` 中存储的 `f1`。

组件重新渲染



重渲染的时候，再次调用 `useCallback` 同样返回给我们 `f1` 函数，并且这个函数还是指向同一块内存，从而使得 `onClick` 函数和上次渲染时真正做到了引用相等。

useCallback 和 useMemo 的关系

之前我们说 `Memoization` 的两大场景

1. 通过缓存计算结果，节省费时的计算
2. 保证相同输入下返回值的引用相等

而 `useCallback` 和 `useMemo` 从 `Memoization` 角度来说

- `useCallback` 主要是为了解决“函数的”引用相等 “**问题，

- `useMemo` 则是一个”全能型选手“，能够同时胜任引用相等和节约计算的任务。

实际上，`useMemo` 的功能是 `useCallback` 的超集。

与 `useCallback` 只能缓存函数相比，`useMemo` 可以缓存任何类型的值（当然也包括函数）。
`useMemo` 的使用方法如下：

```
1 const memoizedValue = useMemo(() =>
2   computeExpensiveValue(a, b),
3   [a, b]
4 );
```

其中第一个参数是一个函数，这个函数返回值的返回值（也就是上面 `computeExpensiveValue` 的结果）将返回给 `memoizedValue`。

因此以下两个钩子的使用是完全等价的：

```
1 useCallback(fn, deps);
2 useMemo(() => fn, deps);
```

useReducer

使用 `useState` 的时候遇到过一个问题：通过 `Setter` 修改状态的时候，怎么读取上一个状态值，并在此基础上修改呢？如果你看文档足够细致，应该会注意到 `useState` 有一个{函数式更新|Functional Update}的用法。

```
1 function Counter({initialCount}) {
2   const [count, setCount] = useState(initialCount);
3   return (
4     <>
5       Count: {count}
6       <button onClick={() => setCount(initialCount)}>Reset</button>
7       <button onClick={() => setCount(prevCount => prevCount - 1)}>-</button>
8       <button onClick={() => setCount(prevCount => prevCount + 1)}>+</button>
9     </>
10  );
11 }
```

传入 `setCount` 的是一个函数，它的参数是之前的状态，返回的是新的状态。熟悉 `Redux` 的朋友马上就指出来了：这其实就是一个 `Reducer` 函数。

useState底层实现原理

在 `React` 的源码中，`useState` 的实现使用了 `useReducer`。在 `React` 源码中有这么一个关键的函数 `basicStateReducer`

```
1 function basicStateReducer(state, action) {  
2   return typeof action === 'function' ? action(state) : action;  
3 }
```

于是，当我们通过 `setCount(prevCount => prevCount + 1)` 改变状态时，传入的 `action` 就是一个 `Reducer` 函数，然后调用该函数并传入当前的 `state`，得到更新后的状态。而我们之前通过**传入具体的值修改状态时**（例如 `setCount(5)`），由于不是函数，所以**直接取传入的值作为更新后的状态**。

传入的 `action` 是一个具体的值（`setCount(xx)`）

上一次状态

520

setCount

当传入 `Setter` 的是一个 `Reducer` 函数的时候：`(setCount(c => c + 1))`

